



SMART HOME

Graduation Project



By

Mahmoud Khamis Farag	Ahmed Ramadan Ahmed
Taher Sabry Taher Farag	Ali Khaled Rauf Al-Rawy
Amr Ahmed Abd-Alaziz	Mahmoud Samy Mohamed
Mahmoud Yasser Mahmoud	Mohamed El-Sayed Hassen
Mahmoud Hamed Abdel-Moneim	
Mostafa Mohamed Abdel-Hamid	

Under Supervision of
Dr. Amna Mahmoud

JUNE 20, 2023

FACTUALY OF COMPUTER AND INFORMATION TECNOLGY
Kafr El-Sheikh University



Acknowledgment

It is a pleasure to acknowledge many people who knowingly and unwittingly helped us, to complete our project. First of all, let us thank God for all the blessings, which carried us through all these years. We express our gratitude to

For here cooperation and encouragement during the completion of this project. Also, we want to thank here our project guide who has always stood by our side and guided, appreciated, and encouraged us to get into more and more ventures. Continuing the same, she enlightened us on the various stages during the development of this project and provided us with many insights and useful examples, which proved to be of immense help in the successful completion of this project. We extend our sincere gratitude to our teachers and guides who made unforgettable contributions.

We thank all the non-teaching staff of our faculty that was always ready to help in whatever way they could.

Abstract

Smart homes are becoming increasingly popular as people seek to automate and optimize their living spaces. A smart home is a home that is equipped with various devices and systems that can be controlled remotely using a smartphone or other smart device. These devices and systems can include everything from thermostats and lighting to security cameras and door locks.

The purpose of this project is to design and implement a smart home system that can be controlled using a smartphone app. The system will include several different components, each of which will be integrated into the overall system to provide a seamless experience for the user.

One of the key components of the system will be the smart thermostat. The thermostat will be able to learn the user's preferences over time and adjust the temperature accordingly. For example, if the user typically likes their home to be cooler at night, the thermostat will automatically adjust the temperature to meet this preference.



Another important component of the system will be the smart lighting. The lighting will be able to be controlled remotely using the app, and will also be able to adjust automatically based on the time of day. For example, the lights may dim automatically as the sun sets, creating a more relaxed and peaceful atmosphere in the home.

The system will also include a smart security system. This will include several different components, such as door locks, motion sensors, and security cameras. The user will be able to monitor the security system remotely using the app, and will also receive alerts if any unusual activity is detected.

In addition to the above components, the system will also include smart appliances such as a smart refrigerator and a smart washing machine. These appliances will be able to communicate with the app, allowing the user to monitor their use and receive alerts when they need to be serviced or replaced.

Overall, the smart home system will provide a seamless and convenient experience for the user. By integrating several different components into a single system, the user will be able to control and monitor their home from anywhere in the world using their smartphone or other smart device.

The implementation of the system will involve several different steps. First, the various components will need to be selected and purchased. Once the components have been acquired, they will need to be installed and configured to work together as a single system.

The next step will be to develop the smartphone app that will be used to control the system. This app will need to be designed to be user-friendly and intuitive, allowing the user to easily control and monitor their home.

Finally, the system will need to be tested and refined to ensure that it is working as intended. This will involve testing the various components and the app, as well as conducting user testing to ensure that the system is meeting the needs of the user.

In conclusion, the smart home system project is an exciting and innovative project that has the potential to transform the way we live. By integrating various components into a single system, the user will be able to control and monitor their home from anywhere in the world using their smartphone or other smart device. This project represents the future of home automation, and has the potential to revolutionize the way we live and interact with our homes.





Contents

Acknowledgement	1
Abstract.....	1
Chapter One	7
1 Introduction	8
1.1 Introduction To Project:	8
1.2 Organization Profile.....	9
1.2.1 Range of expertise includes:.....	9
1.2.2 This project is based on:	9
1.2.3 Speed:	9
1.2.4 Expertise:	9
1.2.5 Services:	9
1.3 Purpose of the project:	9
1.4 The problem with the existing system:.....	10
1.5 Solution of these problems:	11
Chapter Two.....	12
2 System requirement and analysis	12
2.1 Introduction.....	12
2.2 Analysis model	13
2.3 Study of the system.....	13
2.3.1 GUI'S.....	13

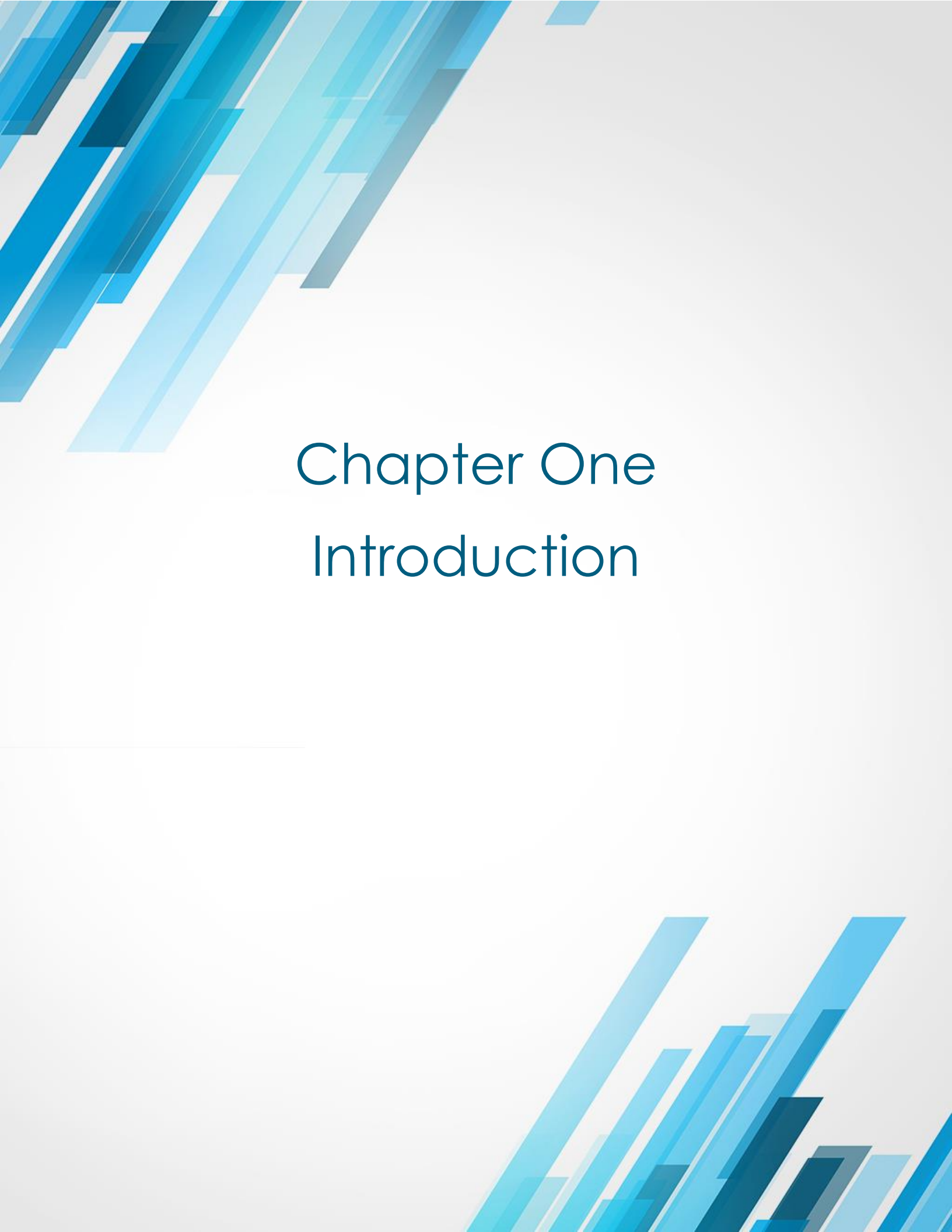


2.3.2	Number of modules	14
2.3.3	Entities Involved in the Project	14
2.4	Hardware specifications.....	15
2.4.1	Hardware requirements:	16
2.4.2	Software requirements:.....	17
2.5	Software requirement specification	18
2.5.1	Introduction to SRS	19
2.5.2	Functional requirements:	20
2.6	Performance requirements	Error! Bookmark not defined.
2.7	Proposed system	Error! Bookmark not defined.
2.7.1	Functional features of the model	Error! Bookmark not defined.
2.7.2	Input and output.....	Error! Bookmark not defined.
Chapter Three		24
3	System Design.....	24
3.1	Introduction.....	24
3.2	E-R Diagram	25
3.3	Data flow diagrams.....	26
3.3.1	DFD Symbols:	27
3.3.2	Constructing A DFD:.....	Error! Bookmark not defined.
3.3.3	Salient features of DFD:.....	Error! Bookmark not defined.
3.3.4	Types of data flow diagrams:	Error! Bookmark not defined.
3.3.5	Rules governing the DFD's	Error! Bookmark not defined.

3.3.6 DFD Diagrams:	Error! Bookmark not defined.
3.4 Unified Modeling Language Diagrams (UML)	36
3.4.1 Architectural views and diagrams of the UML	37
3.4.2 UML Diagrams:.....	38
Chapter Four	39
4 Used Software	40
4.1 Introduction to flutter.....	41
4.2 Features of the common language runtime.....	41
4.3 dart class library.....	42
4.4 What is Dart Programing:	43
4.5 Implementation about Dart Programing:.....	44
4.6 Firebase of Flutter:.....	45
4.7 What about Arduino:.....	46
4.8 Implementation about using Arduino With Flutter.....	47
Chapter Five.....	48
5 System Implementation.....	49
Chapter Six	59
6 System Testing	60
6.1 Introduction.....	61
6.2 Strategic approach to software testing.....	62
6.3 Testing strategy for procedural point of view.....	63
6.4 The levels of testing.....	64



6.5	Methodology	65
Chapter Seven.....		66
6	System Testing	67
6.1	Introduction.....	67
6.2	Client side validation	67
6.3	Server side validation	68
Chapter Eight		69
6	System Testing	70
6.1	Conclusion	70
6.2	Future work.....	70
6.3	Benefits of future work.....	71

The image features a light gray background with abstract, overlapping blue geometric shapes in the top-left and bottom-right corners. These shapes consist of various shades of blue, from light to dark, arranged in a way that creates a sense of depth and movement. The central text is positioned in the middle of the page, between these decorative elements.

Chapter One

Introduction



1.1 Introduction to the project:

With the rapid advancement of technology, the concept of smart home has become increasingly popular in recent years. A smart home is a dwelling that integrates many devices and sensors connected to the Internet to perform and simplify daily tasks, enhance comfort and convenience, improve energy consumption and increase security elements in this type of home.

The purpose of this project is to design a smart home system that meets people's needs and preferences. The system will include various devices such as lighting systems and safety devices such as fire and gas leak detection devices so that they can be tracked and dealt with quickly and correctly, and kitchen utensils that can be controlled remotely via smart phones.

The project will include identifying people's needs and goals, choosing appropriate devices and technologies, and integrating them into one smooth and easy-to-handle system. The system will be tested and improved to ensure that it works properly and does not contain any errors that impede its use at any time and from anywhere we want, whether we are inside or away from the home and meet people's needs.

The ultimate goal of this project is to create a more comfortable, convenient and safe living environment while reducing energy consumption and increasing the sense of satisfaction in our daily lives. The project will also provide valuable insights into the design and implementation of smart home systems, which can be used to inform future projects and developments in this field.

1.2 Organization profile

The scope of experience includes:

In the context of a project, scope of expertise refers to the set of functions and features that a system offers to the user. It includes the various devices and systems that are integrated into the smart home, as well as the control features that are implemented to manage these systems.

Devices that can be integrated into a smart home include lighting systems, heating and cooling systems, security systems, and entertainment systems. It may also include the incorporation of sensors, such as motion sensors and environmental sensors, that can be used to trigger certain actions or perform certain functions.





In addition to hardware and systems integration, the scope of expertise can also include feature implementation. These features enable the user to manage and control various aspects of the home using a single interface, such as a smartphone app. These features can be programmed to automatically perform certain actions, such as turning on lights when someone enters a room or adjusting the temperature based on a specific time.

The way we will deal with the project:

A strategy through which we design, integrate and manage the technology used.

It can be based on different technologies and platforms, depending on the specific requirements and goals of the project. like:

Internet of Things (IoT) platforms:

These platforms provide a framework for connecting and managing various devices and systems in the smart home

Mobile applications: These applications provide the user with a convenient interface to manage and control the smart home

Ultimately, the technologies and platforms used in a smart home project will depend on the goals, requirements, and budget of the project. The goal is to create a seamless and intuitive experience for the user while providing the functionality and features needed to meet their needs.

Speed:

We are aware of the importance of time in our daily lives, and the most important thing is to always search for personal comfort, and this is what is available in smart homes, unlike traditional homes in which we do almost everything by ourselves without the help of anything that can help us do these things. In smart homes, we do it all. In a semi-smooth way, it does not take us time or effort, and it also guarantees speed in implementation.

1.3 Project Purpose:

Some of the benefits of a smart home include energy savings, increased security, improved comfort and convenience, and greater control over home devices and systems. For example, smart thermostats can learn your preferences and adjust the temperature accordingly, resulting in energy savings. Smart home security systems can provide real-time alerts and remote monitoring, giving homeowners peace of mind when they're away from home.

In general, the purpose of a smart home is to improve the quality of life of users by making their homes more comfortable, safe, energy efficient, and convenient.





1.4 Traditional and Smart Home Comparison:

Let's take a quick look at the concept of home automation. Ten minutes after you wake up, your coffee is ready. Home automation is exactly that, home functions are programmed to run automatically without you lifting a finger. In case you thought this was only for lazy people, a smart home can improve efficiency, convenience, and sustainability. You can save time and energy while enjoying the benefits of technology.

Here we will introduce a comparison of traditional and smart homes, including:

There are many problems that users may encounter in traditional homes. Here are some examples:

- **Lack of energy efficiency:** Traditional homes may have outdated or inefficient heating, cooling, lighting and appliance systems, which can lead to higher energy bills and increased carbon emissions.
- **Limited Security:** Traditional homes may rely on traditional locks and alarm systems, which can be easier to bypass or disable than smart security systems.
- **Requires Extra Effort:** Traditional homes may require homeowners to manually perform tasks such as turning on the lights or adjusting the thermostat, which can be time consuming and inconvenient.
- **Problems with tasks:** Access to traditional homes may be difficult for individuals with disabilities or those with mobility issues, as they may require physical manipulation of things such as light switches or thermostats.
- **Maintenance issues:** Traditional homes may be more prone to maintenance problems such as water leaks or mold growth, as homeowners may not be able to detect these problems until they become more serious.

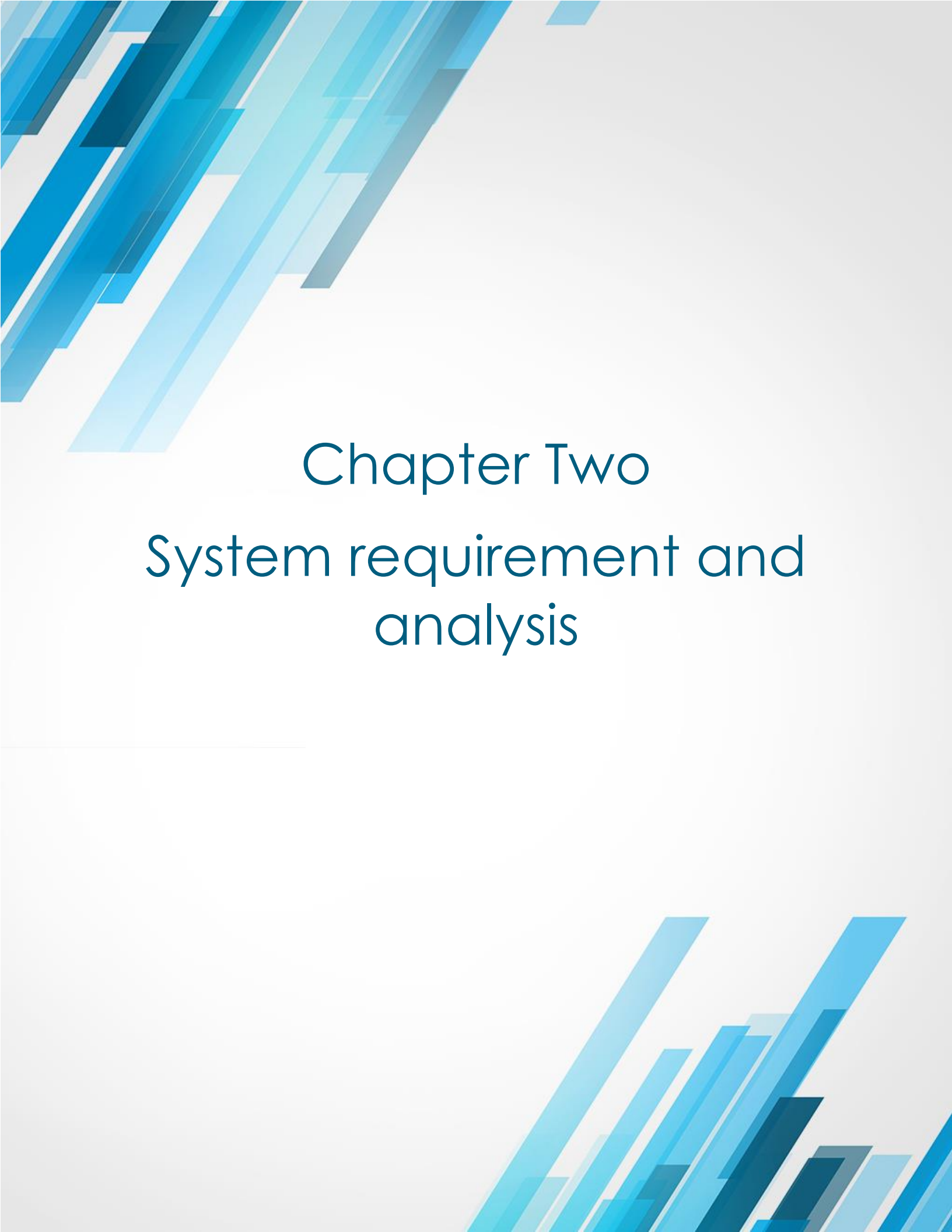


On the contrary, solutions to these problems are found in smart homes:

- There are several ways to solve traditional home problems using smart technology. Here are some examples:
- Install a smart thermostat: The smart thermostat can learn your preferences and automatically adjust the temperature to save energy and increase comfort.
- Use of smart lighting: Smart lighting can be controlled remotely, allowing homeowners to turn lights on and off from their smartphones. This can also save energy by turning off the lights when they are not needed.
- Install a smart security system: Smart security systems can include motion sensors and alarms that can be controlled and monitored remotely. This can provide users with greater peace of mind and improved home security.
- Use of smart appliances: Smart appliances such as refrigerators, washing machines, and ovens can be controlled and monitored remotely, and they can also learn your preferences to improve performance and reduce energy use.

Implement a smart home automation system: A smart home automation system can integrate all your smart devices into a cohesive system, allowing you to control them from a single app or a voice assistant.

This can make everyday tasks more convenient and efficient.



Chapter Two

System requirement and analysis



2.1 Introduction

After analyzing the requirements of the task to be performed, the next step is to analyze the problem and understand its context. The first activity in the phase is studying the existing system and other is to understand the requirements and domain of the new system. Both the activities are equally important, but the first activity serves as a basis of giving the functional specifications and then successful design of the proposed system.

Understanding the properties and requirements of a new system is more difficult and requires creative thinking and understanding of existing running system is also difficult, improper understanding of present system can lead diversion from solution.

2.2 Analysis model

The model that is basically being followed is the WATER FALL MODEL, which states that the phases are organized in a linear order. First of all, the feasibility study is done.

Once that part is over the requirement analysis and project planning begins. If system exists one and modification and addition of new module is needed, analysis of present system can be used as basic model.

The design starts after the requirement analysis is complete and the coding begins after the design is complete. Once the programming is completed, the testing is done. In this model the sequence of activities performed in a software development project are:

- Feasibility study
- Requirement analysis
- Project planning
- System design
- Implementation/coding
- Testing
- Deployment
- Maintenance

Here the linear ordering of these activities is critical. End of the phase and the output of one phase is the input of another phase. The output of each phase is to be consistent with the overall requirement of the system. Some of the qualities of spiral model are also incorporated like after the people concerned with the project review completion of each of the phase the work done.

WATER FALL MODEL was being chosen because all requirements were known beforehand and the objective of our software development is the computerization/automation of an already existing manual working system.

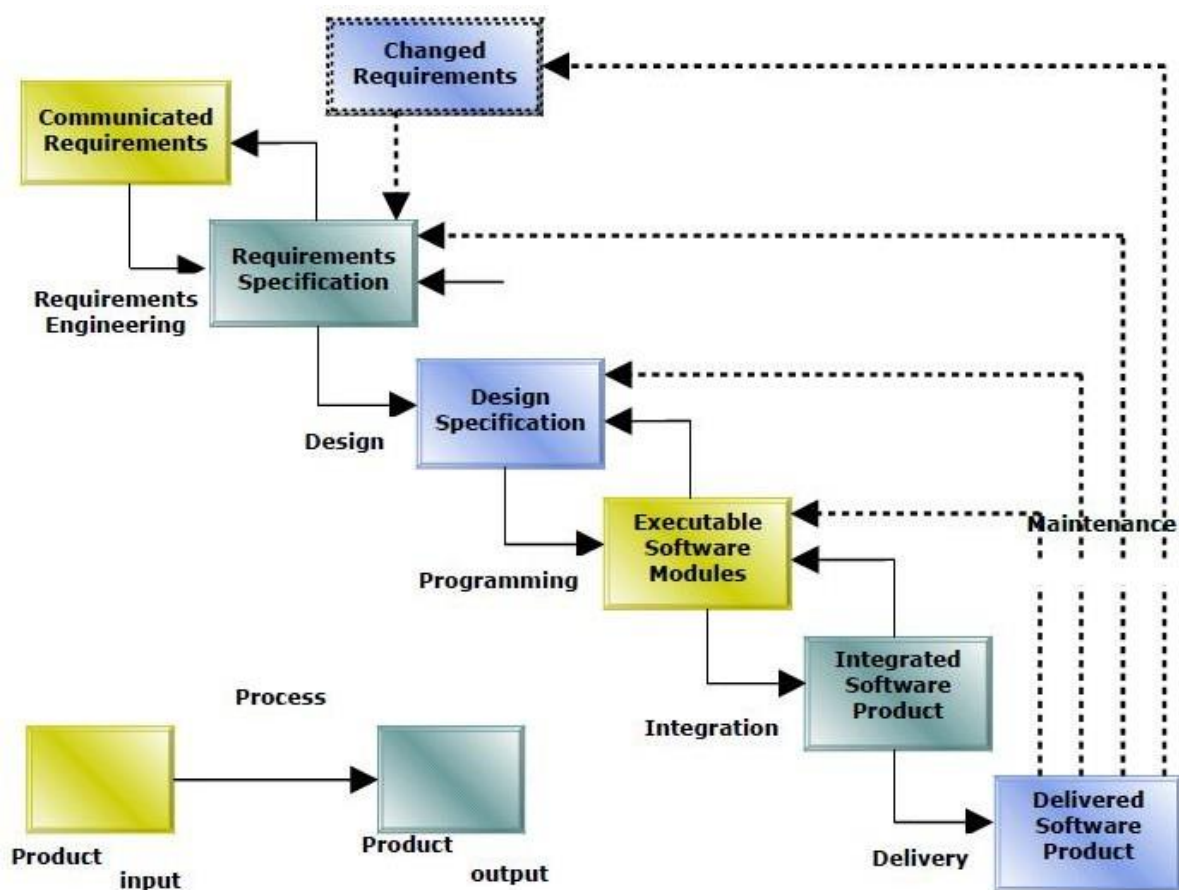


Figure 0-1



2.3 Study of the system

GUI'S

To ensure maximum flexibility for users, the system's user interface has been designed with a graphics-based approach in mind, accessible through a browsing interface. The interface is divided into two main categories: the administrative user interface and the operational or generic user interface.

The administrative user interface is designed to provide consistent information that is an essential part of organizational activities, requiring proper authentication for data collection. This interface is intended to assist administrators with all transactional states, including data insertion, deletion, and updating, as well as providing extensive data search capabilities.

On the other hand, the operational or generic user interface is designed to help users interact with the system in transactions involving existing data and required services. Additionally, the operational user interface enables ordinary users to manage their own information in a customized manner with assisted flexibilities.

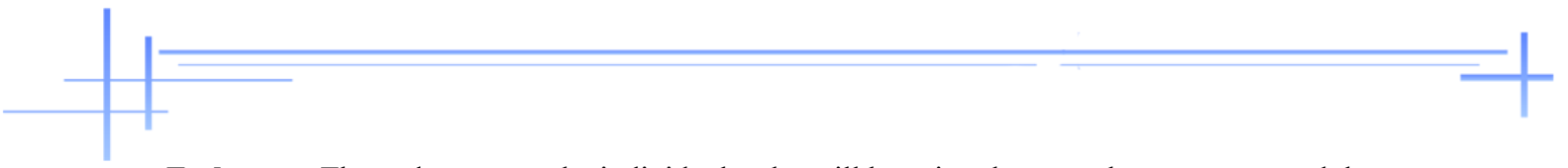
Number of modules

The system after careful analysis has been identified to be presented with the following modules:

- This project is divided into 6 modules:
- Registration Module
- Home automation module
- Security module
- Water sensor module
- Gas sensor module
- Light control module

Entities Involved in the Project

- End-users
- Mobile app stores
- Vendors and suppliers
- Technical support providers
- Payment processing providers



End-users: The end-users are the individuals who will be using the smart home system and the mobile app to control and monitor their homes. They will be the primary beneficiaries of the system and will provide feedback on its usability and functionality.

Mobile app stores: The mobile app will need to be published on various app stores, such as Apple App Store and Google Play, for users to download and install. The app stores may have their own guidelines and requirements for the publication of apps.

Vendors and suppliers: These entities will provide the hardware and software components needed for the smart home system and the mobile app. This may include sensors, automation devices, cameras, and other equipment.

Technical support providers: The project may require technical support providers to provide assistance to end-users with any issues or problems they may encounter while using the smart home system and the mobile app.

Payment processing providers: If the smart home system and the mobile app involve any payment processing, such as for subscription or purchase of additional features, payment processing providers may be involved in the project.

2.4 Hardware specifications

Hardware requirements:

- Smart home devices: The system will require smart home devices such as smart thermostats, smart lights, smart locks, and smart security cameras, which are compatible with the system and can be controlled through the mobile app.
- Sensors: Sensors such as gas sensors, water sensors, and motion sensors may be required to detect any potential hazards and to trigger actions in case of emergencies.
- Hub: A hub may be required to connect and control the smart home devices and sensors.
- Wi-Fi network: A stable and strong Wi-Fi network is necessary for the smart home devices and sensors to communicate with the hub and the mobile app.
- Mobile devices: The users will require mobile devices such as smartphones and tablets to control and monitor the smart home system through the mobile app.

- 
- 
- **Power supply:** Adequate power supply is essential for the smart home devices and sensors to function properly. The power supply may come from electrical outlets or batteries depending on the devices.
 - **Mounting equipment:** Mounting equipment such as screws and brackets may be required to install the smart home devices and sensors in appropriate locations.
 - **Cables and wiring:** Depending on the type of devices and sensors, cables and wiring may be required for the installation and connection of the system components.

Software requirements:

- **Mobile app:** A mobile app will be required to control and monitor the smart home system. The app should be compatible with the operating system of the mobile device and has an easy-to-use interface.
- **Operating system:** The operating system of the mobile device should be compatible with the mobile app and should be updated to the latest version to ensure smooth operation.
- **Backend system:** A backend system will be required to manage the data and communication between the mobile app, smart home devices, and sensors.
- **Cloud services:** Cloud services may be required for remote access to the smart home system, data backup, and storage.
- **Data analytics:** The system may require data analytics software to analyze the data collected by the sensors and provide insights to the users.
- **Security software:** Security software such as antivirus and firewall may be required to protect the system from cyber threats.
- **Firmware updates:** Firmware updates may be required for the smart home devices and sensors to ensure the latest security patches and functionality updates.
- **Programming languages and frameworks:** The development of the software components of the system may require programming languages such as C# and frameworks such as Flutter for mobile app development.





2.5 Software requirement specification

The software, Site Explorer is designed for management of sensors in home from a remote location.

Introduction to SRS

Purpose: The main purpose for preparing this document is to give a general insight into the analysis and requirements of the existing system or situation and for determining the operating characteristics of the system.

Scope: This Document plays a vital role in the development life cycle (SDLC) and it describes the complete requirement of the system. It is meant for use by the developers and will be the basic during testing phase. Any changes made to the requirements in the future will have to go through formal change approval process.

2.6 Functional requirements:

Output design

Outputs from computer systems are required primarily to communicate the results of processing to users. They are also used to provides a permanent copy of the results for later consultation. The various types of outputs in general are:

- External Outputs, whose destination is outside the organization.
- Internal Outputs whose destination is within organization and they are the
- User's main interface with the computer.
- Operational outputs whose use is purely with in the computer department.
- Interface outputs, which involve the user in communicating directly with



Output definition

The outputs should be defined in terms of the following points:

- Type of the output
- Content of the output
- Format of the output
- Location of the output
- Frequency of the output
- Volume of the output
- Sequence of the output

It is not always desirable to print or display data as it is held on a computer. It should be decided as which form of the output is the most suitable.

Output Media:

In the next stage it is to be decided that which medium is the most appropriate for the output. The main considerations when deciding about the output media are:

- The suitability for the device to the particular application.
- The need for a hard copy.
- The response time required.
- The location of the users
- The software and hardware available.

Keeping in view the above description the project is to have outputs mainly coming under the category of internal outputs. The main outputs desired according to the requirement specification are:

"The outputs were needed to be generated as a hot copy and as well as queries to be viewed on the screen. Keeping in view these outputs, the format for the output is taken from the outputs, which are currently being obtained after manual processing. The standard printer is to be used as output media for hard copies".



Input design

Input design is a part of overall system design. The main objective during the input design is as given below:

- To produce a cost-effective method of input.
- To achieve the highest possible level of accuracy.
- To ensure that the input is acceptable and understood by the user.

Input stages:

The main input stages can be listed as below:

- Data recording
- Data transcription
- Data conversion
- Data verification
- Data control
- Data transmission
- Data validation
- Data correction

Input types:

It is necessary to determine the various types of inputs. Inputs can be categorized as follows:

- External inputs, which are prime inputs for the system.
- Internal inputs, which are user communications with the system.
- Operational, which are computer department's communications to the system?
- Interactive, which are inputs entered during a dialogue.



Input media:

At this stage choice has to be made about the input media. To conclude about the input media consideration has to be given to:

- Type of input
- Flexibility of format
- Speed
- Accuracy
- Verification methods
- Rejection rates
- Ease of correction
- Storage and handling requirements
- Security
- Easy to use
- Portability

Keeping in view the above description of the input types and input media, it can be said that most of the inputs are of the form of internal and interactive. As input data is to be directly keyed in by the user, the keyboard can be considered to be the most suitable input device.

Error avoidance

At this stage care is to be taken to ensure that input data remains accurate from the stage at which it is recorded up to the stage in which the data is accepted by the system. This can be achieved only by means of careful control each time the data is handled.

Error detection

Even though every effort is made to avoid the occurrence of errors, still a small proportion of errors is always likely to occur, these types of errors can be discovered by using validations to check the input data.



Data validation

Procedures are designed to detect errors in data at a lower level of detail. Data validations have been included in the system in almost every area where there is a possibility for the user to commit errors. The system will not accept invalid data. Whenever an invalid data is keyed in, the system immediately prompts the user and the user has to again key in the data and the system will accept the data only if the data is correct. Validations have been included where necessary.

The system is designed to be a user friendly one. In other words, the system has been designed to communicate effectively with the user. The system has been designed with popup menus.

Performance requirements

Performance is measured in terms of the output provided by the application.

Requirement specification plays an important part in the analysis of a system. Only when the requirement specifications are properly given, it is possible to design a system, which will fit into required environment. It rests largely in the part of the users of the existing system to give the requirement specifications because they are the people who finally use the system. This is because the requirements have to be known during the initial stages so that the system can be designed according to those requirements. It is very difficult to change the system once it has been designed and on the other hand designing a system, which does not cater to the requirements of the user, is of no use. The requirement specification for any system can be broadly stated as given below:

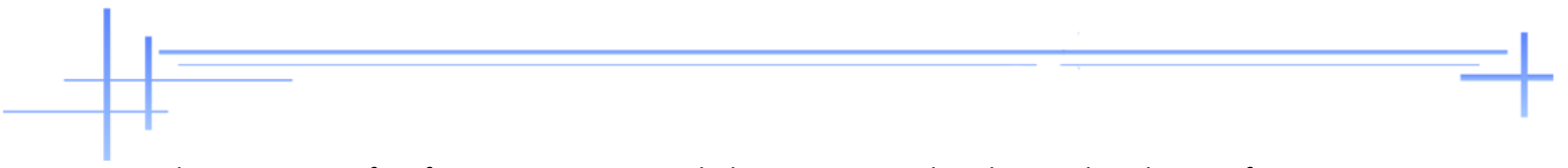
- The system should be able to interface with the existing system
- The system should be accurate
- The system should be better than the existing system

The existing system is completely dependent on the user to perform all the duties.

Proposed system

Functional features of the model

The functionality of the smart home system is designed to be simple and user-friendly, with the goal of enhancing the safety and security of the home. The system is organized into several key modules, including the Mobile App, Gas Sensor, Water Sensor, and Home Automation. The Mobile App serves as



the primary interface for users to interact with the system, providing them with real-time information about their home's status and enabling them to control various features remotely. The Gas Sensor module detects the presence of hazardous gases and alerts the system to take appropriate action to mitigate the risk, such as shutting off the gas source and notifying the user. The Water Sensor module monitors for leaks and other water-related issues, allowing the system to respond quickly and prevent potential damage to the home.

Finally, the Home Automation module offers a range of customizable features, allowing users to program their home's various systems to operate automatically based on their preferences. Together, these modules provide a comprehensive solution for smart home management, offering users peace of mind and increased control over their living environment.

Input and output

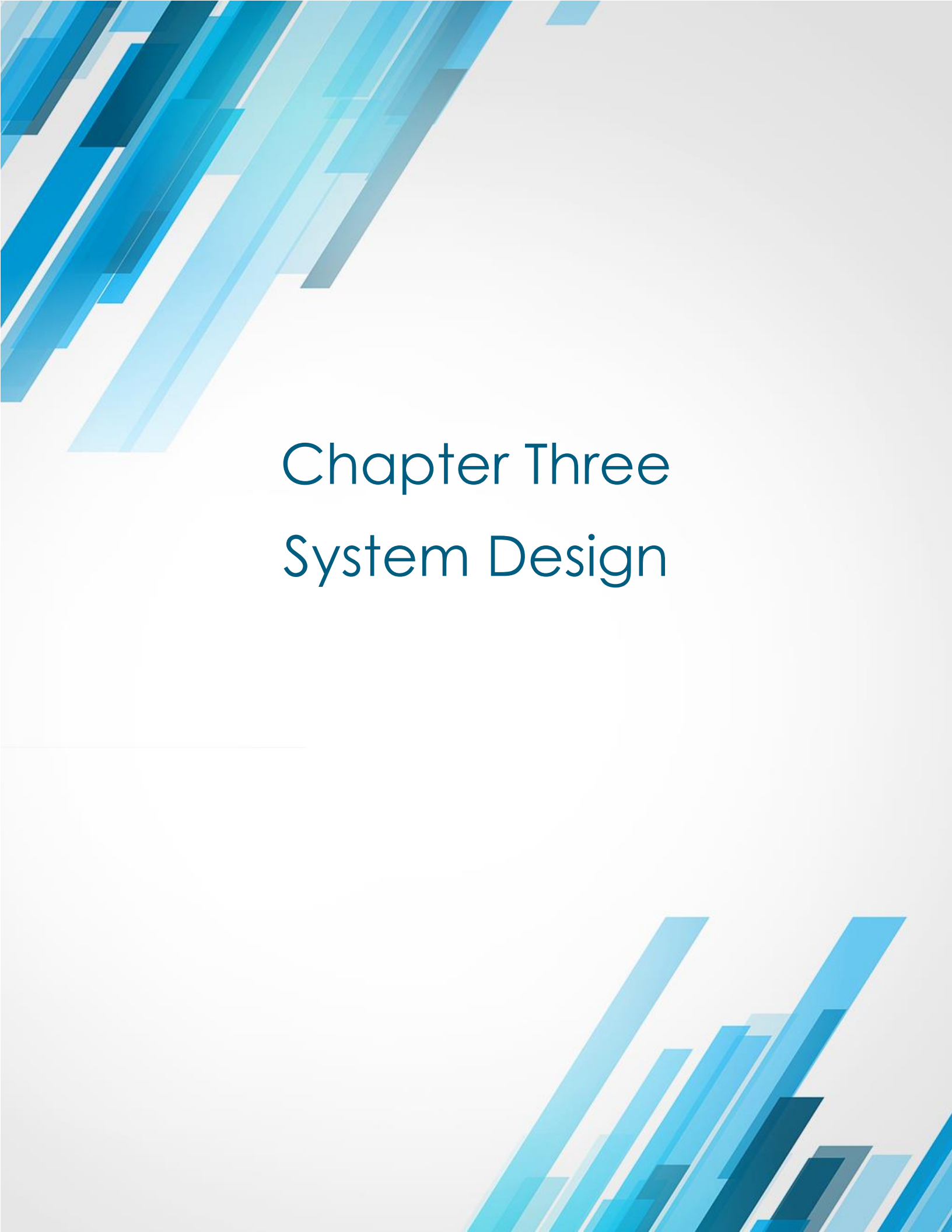
The main inputs, outputs and major functions of the system are as follows

Inputs:

- User enters his or her username and password.
- User sends a command via the mobile app to turn on the lights in a specific room.
- Gas sensor detects a gas leak in the kitchen.
- Water sensor detects water leakage in the basement.
- User sets alerts for when gas or water sensors detect a problem.
- User sets security settings for doors and windows.

Outputs:

- System turns on the lights in the specified room.
- System alerts the user via the mobile app about the gas leak and automatically shuts off the gas source.
- System alerts the user via the mobile app about the water leak and automatically shuts off the water source.
- System sends a notification to the user about the status of the home (e.g., safe or unsafe).
- System sends notifications to user when gas or water sensors detect a problem.
- System sends notifications to user when security settings are triggered (e.g., door or window opened).

The background features abstract, overlapping blue geometric shapes in the top-left and bottom-right corners, creating a modern, architectural feel. The shapes are in various shades of blue, from light to dark, and are oriented diagonally.

Chapter Three

System Design



3.1 Introduction

A smart home is a modern home that uses automation and interconnected devices to provide enhanced convenience, comfort, and security to its inhabitants. The system design for a smart home involves integrating different devices and technologies to create a seamless user experience.

Here are some of the key components of a smart home system design:

Connectivity: Smart homes rely on a robust network of devices and sensors that are interconnected through a wireless network. The network should be designed to handle high data traffic and support communication between devices in real-time.

Sensors: A variety of sensors can be used in a smart home to detect changes in the environment and trigger automated responses. Common sensors include motion sensors, temperature sensors, light sensors, and humidity sensors.

Controllers: Controllers are used to manage and direct the actions of different devices in a smart home. They can be programmed to respond to certain inputs or triggers and execute specific commands.

Devices: Smart home devices can include everything from smart thermostats and lighting systems to security cameras and voice assistants. These devices can be controlled remotely through a mobile app or a central hub.

Data storage and analysis: Smart homes generate a lot of data that can be used to improve the user experience and optimize the system. Data can be collected and analyzed to identify patterns, optimize energy usage, and improve security.

Overall, a well-designed smart home system should be easy to use, reliable, and secure. It should provide the user with complete control over their home environment while minimizing energy consumption and ensuring the safety of the occupants.



3.2 E-R Diagram

An E-R (Entity-Relationship) diagram is a visual representation of the relationships between various entities in a system. In the case of a smart home, entities could include devices, sensors, controllers, users, and other components of the system.

Here's an example of an E-R diagram for a smart home system design:

In this diagram, we have several entities represented by rectangles:

User: Represents the user of the smart home system.

Device: Represents a smart home device, such as a thermostat, lighting system, or security camera.

Sensor: Represents a sensor that is used to detect changes in the environment, such as a motion sensor or temperature sensor.

Controller: Represents a controller that is used to manage and direct the actions of different devices in the smart home system.

Room: Represents a room in the house where devices and sensors may be located.

The relationships between these entities are represented by lines and symbols:

Device-Sensor Relationship: A device can have one or more sensors associated with it. This is represented by the diamond symbol on the Device side of the line.

Device-Controller Relationship: A device can be controlled by one or more controllers. This is represented by the diamond symbol on the Device side of the line.

Room-Device Relationship: A device can be located in one or more rooms. This is represented by the line between Room and Device.

User-Controller Relationship: A user can have one or more controllers associated with their account. This is represented by the line between User and Controller.

This E-R diagram provides a high-level view of the different components of a smart home system and how they are related to each other. It can be used as a starting point for further system design and development.



3.3 Data flow diagrams

A data flow diagram (DFD) is a visual representation of the flow of data between different components of a system. In the case of a smart home, data could include sensor readings, user inputs, and system outputs.

Here's an example of a DFD for a smart home system design:

In this diagram, we have several components represented by rectangles:

Sensors: Represented by the "Sensors" rectangle, these devices detect changes in the environment and provide data to the system.

Controllers: Represented by the "Controllers" rectangle, these devices receive data from sensors and use it to control smart home devices.

Devices: Represented by the "Devices" rectangle, these are the smart home devices that are controlled by the controllers.

User Interface: Represented by the "User Interface" rectangle, this is where the user interacts with the smart home system, providing input and receiving output.

The arrows between the components represent the flow of data between them:

Sensor Data Flow: Data flows from the sensors to the controllers, providing information about the current state of the environment.

Controller Data Flow: Data flows from the controllers to the devices, telling them what actions to take based on the sensor data.

User Input Data Flow: Data flows from the user interface to the controllers, allowing the user to control the smart home devices.

User Output Data Flow: Data flows from the devices to the user interface, providing feedback to the user about the state of the system.

This DFD provides a detailed view of how data flows through a smart home system, from the sensors to the devices and the user interface. It can be used to identify potential bottlenecks or areas for optimization in the system design.

DFD Symbols:

In the DFD, there are four symbols:

- A square defines a source(originator) or destination of system data.
- An arrow identifies data flow. It is the pipeline through which the information flows.
- A circle or a bubble represents a process that transforms incoming data flow into outgoing data flows.
- An open rectangle is a data store, data at rest, or a temporary repository of data.

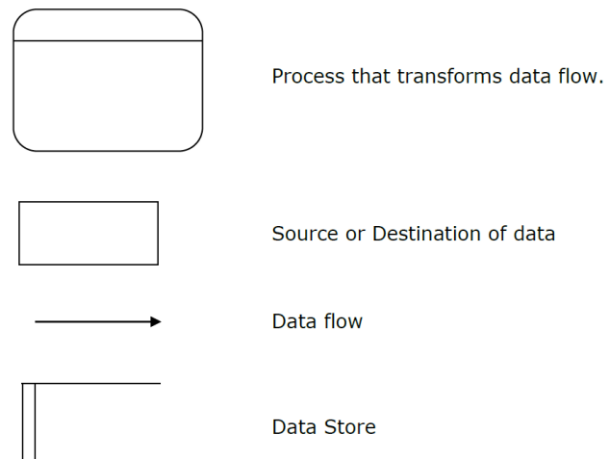


Figure 0-1

Constructing A DFD:

constructing a DFD (Data Flow Diagram) for a smart home system design involves identifying the main components of the system, identifying the data inputs and outputs for each component, and creating a diagram that accurately represents the data flows between components. The DFD can be refined through testing and iteration and should be documented for future reference.



Salient features of DFD:

Salient features of a DFD (Data Flow Diagram) in system design for a smart home include:

Identifying the main components of the system and their data inputs and outputs.

Representing data flows between components using arrows and symbols.

Providing a visual representation of the flow of data through the system.

Enabling the identification of potential bottlenecks and areas for optimization.

Providing a basis for testing and iteration to refine the system design.

Documenting the system design for future reference.

Types of data flow diagrams:

- Current Physical
- Current Logical
- New Logical
- New Physical

Rules governing the DFD's

There are some general rules that govern the creation of DFDs (Data Flow Diagrams) for a smart home system design. Here are a few:

1. Consistency: The notation and symbols used in the DFD should be consistent throughout the diagram and adhere to standard conventions.
2. Completeness: The DFD should include all components of the smart home system and show all data flows between them.
3. Clarity: The DFD should be easy to read and understand, with clear labels and symbols that accurately represent the components and data flows.



4. Modularity: The DFD should be modular, with each component represented as a separate entity and each data flow represented as a separate arrow.

5. Minimization: The DFD should be as simple as possible while still accurately representing the smart home system.

6. Validation: The DFD should be validated through testing and iteration to ensure that it accurately represents the smart home system and can be used for further development.

By following these rules, a well-designed DFD can provide a clear and accurate representation of the smart home system and its data flows. This can help identify potential issues and areas for optimization and can be used as a basis for further system design and development.

Process:

Identify the main components of the smart home system: This includes sensors, controllers, devices, and the user interface.

Identify the data inputs and outputs for each component: This includes the data that each component receives and produces, such as sensor readings, user inputs, and system outputs.

Data Store:

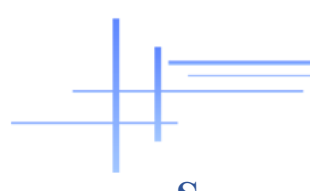
Clearly label data stores: Data stores should be clearly labeled and identified in the DFD to indicate what type of data is stored in them.

Show data flows to and from data stores: Data flows to and from data stores should be shown using arrows and symbols to indicate the direction of data flow.

Show access modes for data stores: Access modes, such as read, write, or delete, should be shown for data stores to indicate how data is accessed and modified.

Minimize the number of data stores: The number of data stores in the DFD should be minimized to keep the diagram simple and easy to understand.

Ensure consistency: The notation and symbols used for data stores should be consistent throughout the DFD and adhere to standard conventions.



Source or sink:

The origin and /or destination of data.

- Data cannot move directly from a source to sink it must be moved by a process
- A source and /or sink has a noun phrase label

Data Flow:

- A Data Flow has only one direction of flow between symbols. It may flow in both directions between a process and a data store to show a read before an update. The latter is usually indicated however by two separate arrows since these happen at different times.
- A join in DFD means that exactly the same data comes from any of two or more different processes, data store or sink to a common location.
- A data flow cannot go directly back to the same process it leads. There must be at least one other process that handles the data flow, produce some other data flow, returns the original data into the beginning process.
- A Data flow to a data store means update (delete or change). 5. A data Flow from a data store means retrieve or use.

A data flow has a noun phrase label more than one data flow noun phrase can appear on a single arrow as long as all of the flows on the same arrow move together as one package.



DFD Diagrams:

Login Diagram

Login DFD:

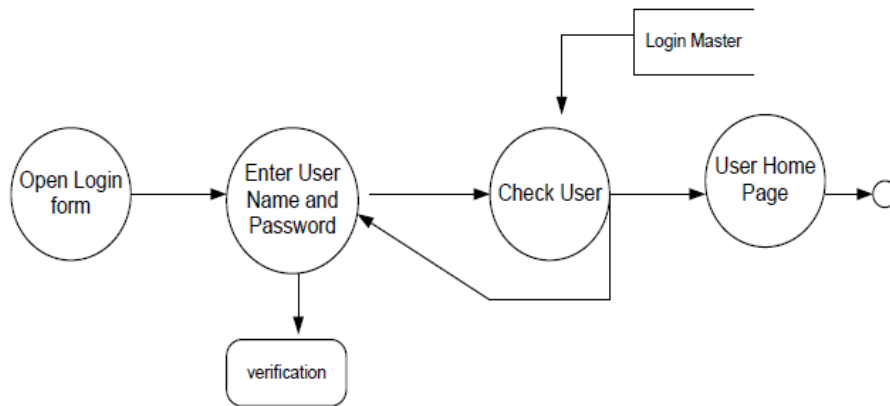


Figure 0-2

User

Activities DFD

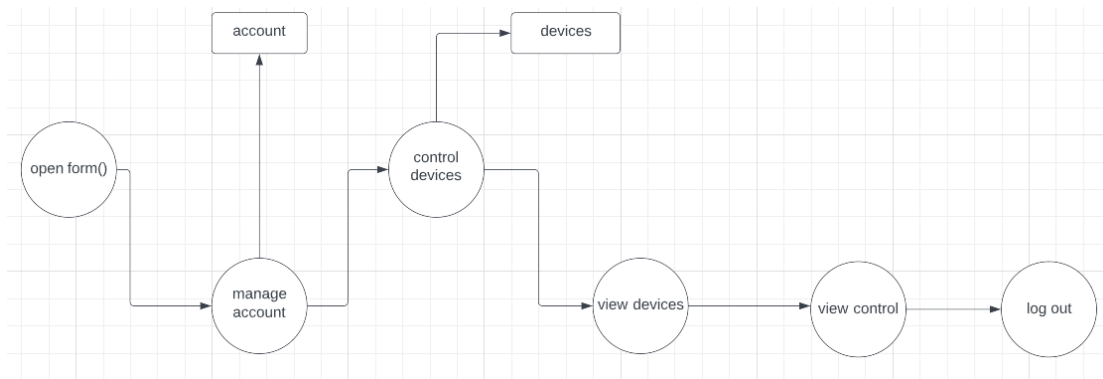


Figure 0-5



Unified Modeling Language Diagrams (UML)

Building a model for a software system prior to its construction is as essential as having a blueprint for building a large building. Good models are essential for communication among project teams. As the complexity of the systems increases, so does the importance of good modeling techniques.

The use of visual notation to represent or model a problem can provide us several benefits relating to clarity, familiarity, maintenance, and simplification. The main reason for modeling is the reduction of complexity.

The Unified Modeling Language (UML) is a set of notations and conventions used to describe and model an application.

The UML is intended to be a universal language for modeling systems, meaning that it can express models of many different kinds and purposes, just as a programming language or a natural language can be used in different ways. A "model" is an abstract representation of a system, constructed to understand the system prior to building or modifying it.

The term "system" is used here in a broad sense to include any process or structure. For example, the organizational structure of a corporation, health services, computer software, instruction of any sort (including computers), the national economy, and so forth all would be termed "systems".

The unified modeling language is a language for specifying, constructing, visualizing, and documenting the software system and its components.

The UML is a graphical language with sets of rules and semantics. The rules and semantics of a model are expressed in English, in a form known as "object constraint language"(OCL).

OCL is a specification language that uses simple logic for specifying the properties of a system.

The UML is not intended to be a visual programming language in the sense of having all the necessary visual and semantic support to replace programming languages.

However, the UML does have a tight mapping to a family of object-oriented languages, so that you can get the best of both worlds.

The primary goals in the design of the UML were as follows:



- 
- 
- Provide users ready-to-use, expensive visual modeling languages so they can develop and exchange meaningful models.
 - Provide extendibility and specialization mechanisms to extend the core concepts.
 - Be independent of particular programming languages and development process.
 - Provide a formal basis for understanding the modeling language.
 - Encourage the growth of the OO tools market.
 - Support higher level development concepts.
 - Integrate best practices and methodologies.

UML is a language used to:

- Visualize the software system well-defined symbols. Thus, a developer or tool can unambiguously interpret a model written by another developer, using UML
- Specify the software system and help building precise, unambiguous and complete models.
- Construct the models of the software system that can directly communicate with a variety of programming languages.
- Document models of the software system during its development stages.

3.4 Architectural views and diagrams of the UML

The UML Meta model elements are organized into diagrams. Different diagrams are used for different purposes depending on the angle from which you are viewing the system. The different views are called architectural views. Architectural views facilitate the organization of knowledge, and diagrams enable the communication of knowledge. Then knowledge itself is within the model or set of models that focuses on the problem and solution. The architectural views and their diagrams are summarized below:





The user model view

Encompasses a problem and solution from the perspective of those individuals whose problem the solution addresses. The view presents the goals and objectives of the problem owners and their requirements of the solution. This view is composed of use case diagrams. These diagrams describe the functionality provided by a system to external interactors. These diagrams contain actors, use cases, and their relationships.

The Structural model views

Encompasses the static, or structural, aspects of a problem and solution. This view is also known as the static or logical view. This view is composed of the following diagrams:

- **Class diagrams:** describe the static structure of a system, or how it is declared rather than how it behaves. These diagrams contain classes and associations.
- **object diagrams:** describe the static structure of a system at a particular time during its life. These diagrams contain objects and links.
-

The behavioral model views

Encompasses the dynamic or behavioral aspects of a problem and solution. The view is also known as the dynamic, process, concurrent or collaborative view. This view is composed of the following diagrams:

- **Sequence diagrams:** render the specification of behavior. These diagrams describe the behavior provided by a system to interactions. These diagrams contain classes that exchange messages with in an interaction arranged in time sequence. In generic form, these diagrams describe a set of message exchange sequences among a set of classes. In instance form(scenarios), these diagrams describe one actual message exchange sequence among objects of those classes.
- **Collaboration diagrams:** render how behavior is realized by components with in a system. These diagrams contain classes, associations, and their message exchanges with in a collaboration to accomplish a purpose. In generic form, these



diagrams describe a set of classes and associations involved in message exchange sequences. In instance form(scenarios), these diagrams describe a set of objects of those classes' links confirming to the associations, and one actual message exchange sequence that inconsistent with the generic form and uses those objects and links.

- **State chart diagrams:** render the states and responses of a class participating in behavior, and the life cycle of an object. These diagrams describe the behavior of a class in response to external stimuli.

- **Activity diagrams:** render the activities of a class participating in behavior. These diagrams describe the behavior of a class in response to internal processing rather than external events. Activity diagrams describe the processing activities with in a class.

- **The Implementation model view:** Encompasses the structural and behavioral aspects of the solution's realization. This view is also known as the component or development view and is composed of component diagrams. These diagrams describe the organization of and dependencies among software implementation components. These diagrams contain components and their relationships.

- **The Environment model view:** Encompasses the structural and behavioral aspects of the domain in which a solution must be realized. This view is also known as the deployment or physical view. This view is composed of deployment diagrams. These diagrams describe the configuration of processing resources elements and the mapping of software implementation components onto them. These diagrams contain nodes, components and their relationships.

3.5 UML Diagrams

Use Case Diagram:

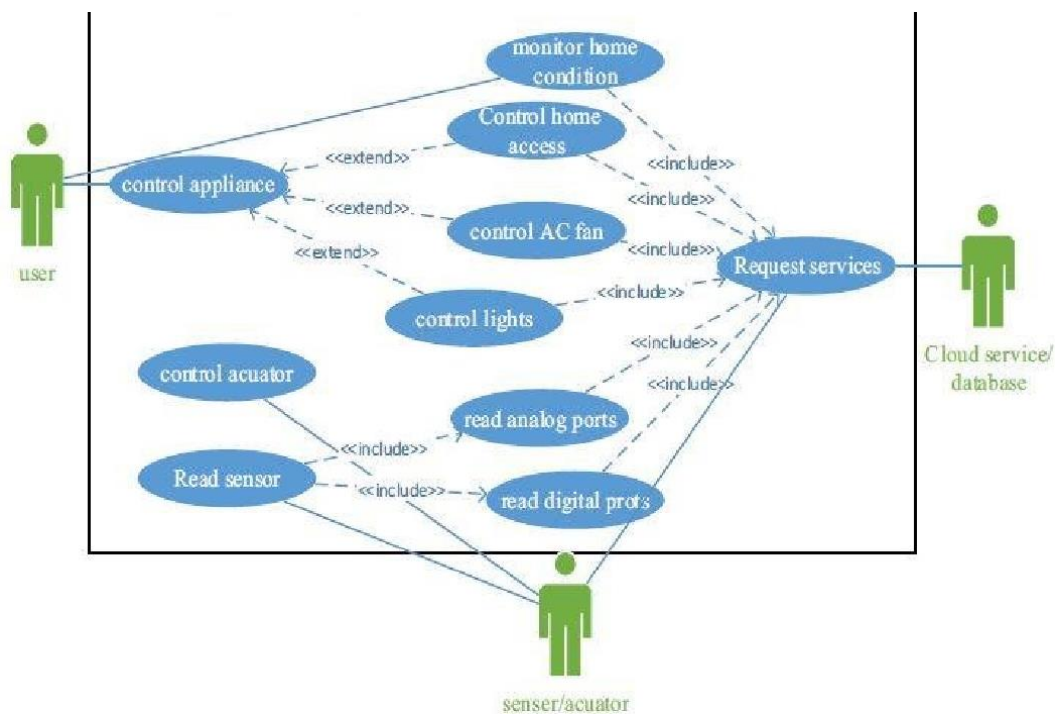
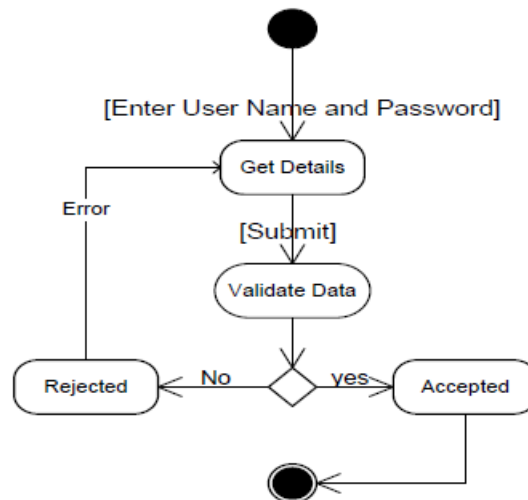


Figure 0-6 Use Case for Admin and User of App

Activity Diagram:

Login Activity:



Registration Activity

Figure 0-7 Activity Diagram for Login

Diagram

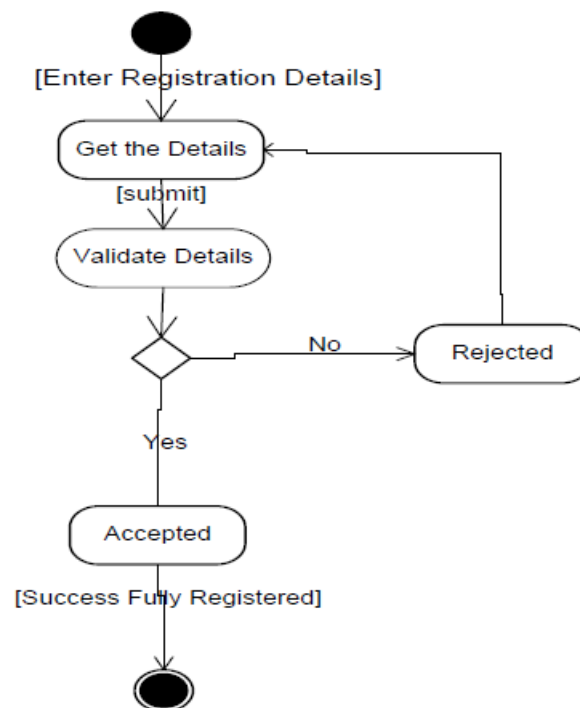
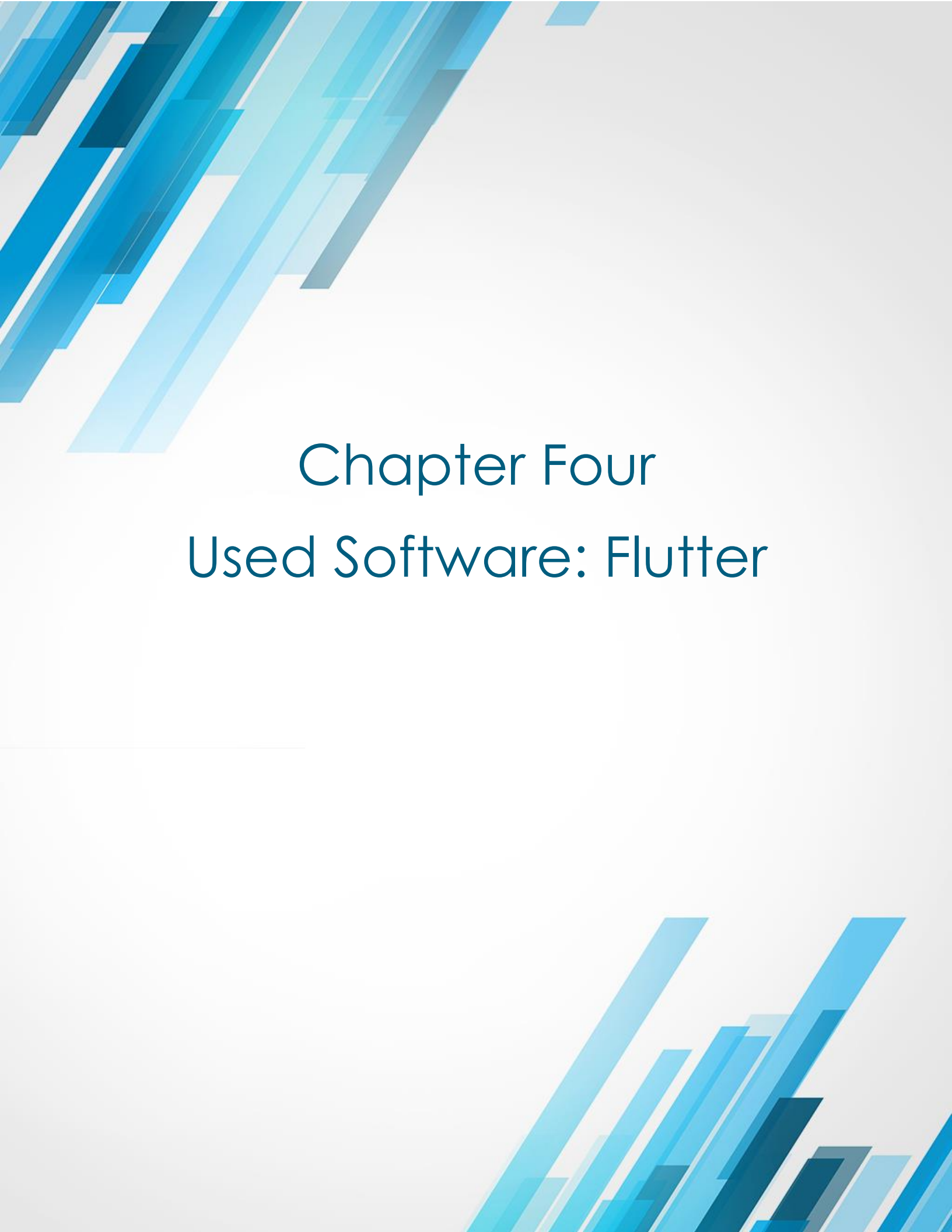
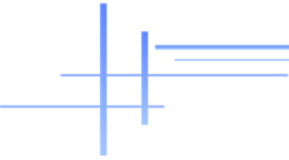


Figure 0-8 Activity Diagram for Registration



Chapter Four

Used Software: Flutter



4.1 Flutter – Introduction

Flutter is an open-source UI framework developed by Google that allows developers to create high-performance, beautiful, and natively compiled applications for mobile, web, and desktop platforms using a single codebase. With Flutter, developers can build visually attractive and responsive apps that run smoothly on both Android and iOS devices, while also having the ability to create web and desktop applications with the same code.

However, to develop an application supporting both the OSs, we need to code in two different languages using two different frameworks. To help overcome this complexity, there exists mobile frameworks supporting both OS.

These frameworks range from simple HTML based hybrid mobile application framework (which uses HTML for User Interface and JavaScript for application logic) to complex language specific framework (which do the heavy lifting of converting code to native code).

Irrespective of their simplicity or complexity, these frameworks always have many disadvantages, one of the main drawback being their slow performance compared with native framework.

In this scenario, Flutter – a simple and high performance framework based on Dart language, provides high performance by rendering the UI directly in the operating system's canvas rather than through native framework.

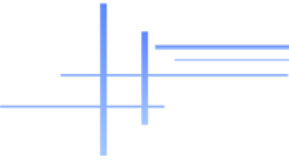
Flutter also offers many ready to use widgets (UI) to create a modern application. These

widgets are optimized for mobile environment and designing the application using widgets is as simple as designing HTML.

Overall, Flutter is a powerful framework that allows developers to create high-quality and visually appealing applications for multiple platforms with ease.

Its simplicity, speed, and flexibility make it a popular choice among developers and businesses looking to build cross-platform applications.





4.2 Features of Flutter

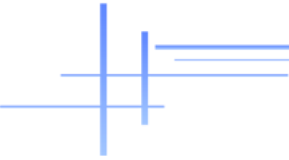
Flutter framework offers the following features to developers:

- Modern and reactive framework.
- Uses Dart programming language and it is very easy to learn.
- Fast development.
- Beautiful and fluid user interfaces.
- Huge widget catalog.
- Runs same UI for multiple platforms.
- High performance application.

4.3 Advantages of Flutter

Flutter comes with beautiful and customizable widgets for high performance and outstanding mobile application. It fulfills all the custom needs and requirements. Besides these, Flutter offers many more advantages as mentioned below:

- Dart has a large repository of software packages which lets you to extend the capabilities of your application.
- Developers need to write just a single code base for both applications (both Android and iOS platforms). Flutter may be extended to other platform as well in the future.
- Flutter needs lesser testing. Because of its single code base, it is sufficient if we write automated tests once for both the platforms.
- Flutter's simplicity makes it a good candidate for fast development. Its customization capability and extendibility makes it even more powerful.
- With Flutter, developers have full control over the widgets and its layout.
- Flutter offers great developer tools, with amazing hot reload.



4.4 Disadvantages of Flutter

Despite its many advantages, flutter has the following drawbacks in it:

- Since it is coded in Dart language, a developer needs to learn new language (though it is easy to learn).
- Modern framework tries to separate logic and UI as much as possible but, in Flutter, user interface and logic is intermixed. We can overcome this using smart coding and using high level module to separate user interface and logic.
- Flutter is yet another framework to create mobile application. Developers are having a hard time in choosing the right development tools in hugely populated segment. Flutter 3 This chapter will guide you through the installation of Flutter on your local computer in detail.

4.5 Architecture of Flutter Application

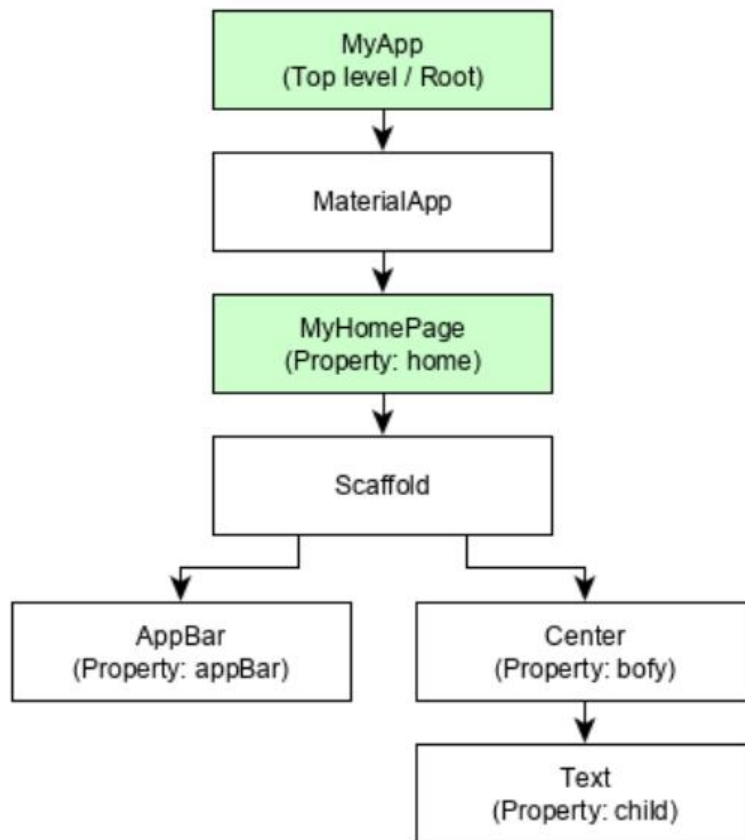
widgets The core concept of the Flutter framework is In Flutter, Everything is a widget. Widgets are basically user interface components used to create the user interface of the application.

In Flutter, the application is itself a widget. The application is the top- level widget and its UI is build using one or more children (widgets), which again build using its children widgets.

This composability feature helps us to create a user interface of any complexity.

For example :





- MyApp is the user created widget and it is build using the Flutter native widget, MaterialApp.
- MaterialApp has a home property to specify the user interface of the home page, which is again a user created widget, MyHomePage.
- MyHomePage is build using another flutter native widget, Scaffold.
- Scaffold has two properties – body and appBar.
- body is used to specify its main user interface and appBar is used to specify its header user interface.
- Header UI is build using flutter native widget, AppBar and Body UI is build using Center widget.
- The Center widget has a property, Child, which refers the actual content and it is build using Text widget.



4.6 Dart Programing

Dart is an object-oriented, class-based programming language that was developed by Google in 2011. It was designed to be easy to learn, fast, and efficient, with a focus on scalability and performance. Dart is primarily used for building web and mobile applications, but it can also be used for server-side programming and command-line tools.

Here are some key features of Dart:

Strong typing: Dart is a statically typed language, which means that variables have a specific data type that is determined at compile time. This helps catch errors early in the development process and improves the performance of the application.

Just-in-time (JIT) and ahead-of-time (AOT) compilation: Dart supports both JIT and AOT compilation, which allows developers to choose the best approach for their application's performance needs. JIT compilation allows for faster development and testing, while AOT compilation is better for production environments where performance is critical.

Garbage collection: Dart's garbage collector automatically manages memory, freeing up unused memory and preventing memory leaks.

Asynchronous programming: Dart supports asynchronous programming, which allows multiple tasks to be executed simultaneously, improving the performance and responsiveness of the application.

Isolates: Dart's isolates are lightweight threads that can run concurrently and communicate with each other using message passing. This allows developers to write concurrent and parallel code with ease.

4.7 Implementation of Dart Programing

Dart can be implemented in a variety of ways, depending on the type of application you want to build. Here are some common ways to implement Dart:

Command-line tools: Dart can be used to write command-line tools that can be executed in a terminal or console. To create a command-line tool, you can use the Dart SDK, which provides a set of command-line tools for compiling and running Dart code.

Server-side programming: Dart can be used to build server-side applications that run on a web server. To build server-side applications, you can use a web framework like Aqueduct, which provides a set of tools and libraries for building RESTful APIs and web applications.

Web development: Dart can be used to build web applications that run in a browser. To build web applications, can use the Flutter framework to build web applications, which allows you to write code once and deploy it to multiple platforms.





Mobile development: Dart can be used to build mobile applications for Android and iOS using the Flutter framework. Flutter allows you to build high-performance, visually attractive, and natively compiled mobile applications using a single codebase. Flutter provides a rich set of pre-built widgets and tools to make building mobile applications faster and easier.

To implement Dart, you will need to install the Dart SDK and a code editor or Integrated Development Environment (IDE). There are many options available, including Visual Studio Code, IntelliJ IDEA, and Android Studio. Once you have installed the necessary tools, you can start writing code in Dart and building your application.

4.8 Firebase of Flutter

Firebase is a mobile and web application development platform that provides a wide range of cloud-based services, including authentication, real-time databases, storage, and analytics. Firebase is owned by Google and provides a powerful set of tools and services for building high-quality applications.

Firebase can be used with Flutter to build scalable and efficient mobile applications. Flutter provides a set of plugins and libraries that make it easy to integrate Firebase into your application. Here are some of the key Firebase services that can be used with Flutter:

Authentication: Firebase Authentication provides a simple and secure way to authenticate users in your application. With Flutter, you can use the Firebase Authentication plugin to handle user authentication and authorization.

Real-time database: Firebase Realtime Database is a cloud-hosted database that allows you to store and sync data in real-time. With Flutter, you can use the Firebase Realtime Database plugin to read and write data to the database in real-time.

Storage: Firebase Storage provides a simple and secure way to store and retrieve user-generated content, such as images and videos. With Flutter, you can use the Firebase Storage plugin to upload and download files to and from Firebase Storage.

Cloud messaging: Firebase Cloud Messaging allows you to send notifications and messages to users in your application. With Flutter, you can use the Firebase Cloud Messaging plugin to send push notifications to users on Android and iOS devices.

Analytics: Firebase Analytics provides insights into user behavior and engagement in your application. With Flutter, you can use the Firebase Analytics plugin to track user events and behavior in your application.

Overall, Firebase provides a powerful set of tools and services that can be used with Flutter to build high-quality mobile applications. By leveraging the power of Firebase, developers can build scalable and efficient applications that provide a great user experience.



4.9 Arduino – Introduction

Arduino is an open-source platform used for building electronic projects. It consists of a microcontroller board, a development environment, and a community of developers and enthusiasts who share projects and knowledge. Arduino is designed to be easy to use, even for people with no prior experience in electronics or programming.

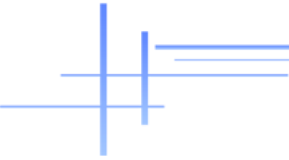
The Arduino board contains a microcontroller, which is a small computer that can be programmed to perform specific tasks. The board also contains input and output pins, which can be used to connect sensors, actuators, and other electronic components. The development environment provides a user-friendly interface for writing, compiling, and uploading code to the board.

Arduino is popular among hobbyists, students, and professionals who want to build interactive and creative projects. Some common applications of Arduino include robotics, home automation, wearable technology, and Internet of Things (IoT) devices. Arduino is also used in education to teach programming, electronics, and physical computing.

Arduino has a large and active community of developers and enthusiasts who share code, tutorials, and projects through online forums and websites. This community has contributed to the development of hundreds of libraries and tools that make it easier to build projects with Arduino.

Overall, Arduino is a versatile and accessible platform that can be used to build a wide range of electronic projects. Its simplicity, affordability, and community support have made it a popular choice among hobbyists, students, and professionals alike.





4.10 Using Arduino With Flutter

Using Arduino with Flutter can be an effective way to create innovative and interactive projects. Arduino is an open-source microcontroller platform that allows you to create and control electronic devices through programming. Flutter, on the other hand, is an open-source mobile application development framework that enables you to create high-performance and visually appealing cross-platform apps.

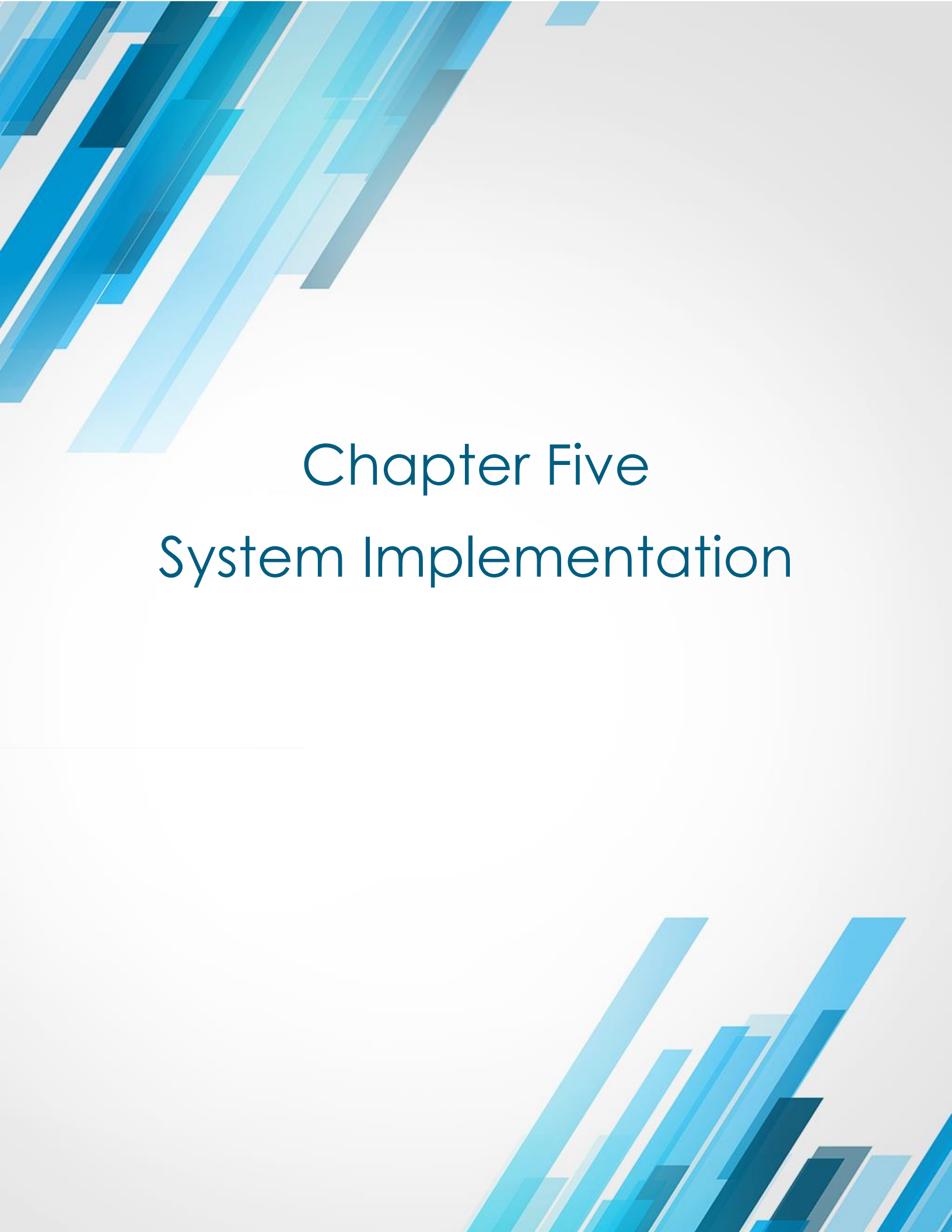
Combining the two technologies can allow you to create a wide range of applications, from home automation systems to smart wearables. By using Arduino with Flutter, you can take advantage of the powerful features and capabilities of both technologies.

One of the biggest advantages of using Arduino with Flutter is the ease of integration. The two platforms work well together, and there are many libraries and plugins available that can help you connect them seamlessly. Additionally, Flutter's Hot Reload feature makes it easy to test and debug your Arduino code, allowing you to quickly iterate and improve your project.

Another advantage of using these two technologies together is the flexibility it provides. With Arduino, you can control sensors, motors, and other electronic devices, while Flutter can provide a user-friendly interface for your application. This allows you to create complex and interactive projects that can respond to user input in real-time.

Overall, using Arduino with Flutter can be a powerful combination that opens up many possibilities for creating innovative and interactive projects. Whether you are a hobbyist or a professional, this combination can provide a flexible and effective solution for your needs.





Chapter Five

System Implementation

12:27

20.0 KB/S



Smart Connect

Control your home easier



12:27

64.0 KB/S



Register

Register Now To Control The Home



Name



Email



*** Password



Phone

SignUp

12:27

8.00 KB/S

Welcome,
Sign in to Continue

SignUp

 Email

*** Password



[Forgot Password?](#)

SignIn

__ OR __



Sign In with Facebook



Sign In with Google

12:27

10.0 KB/S

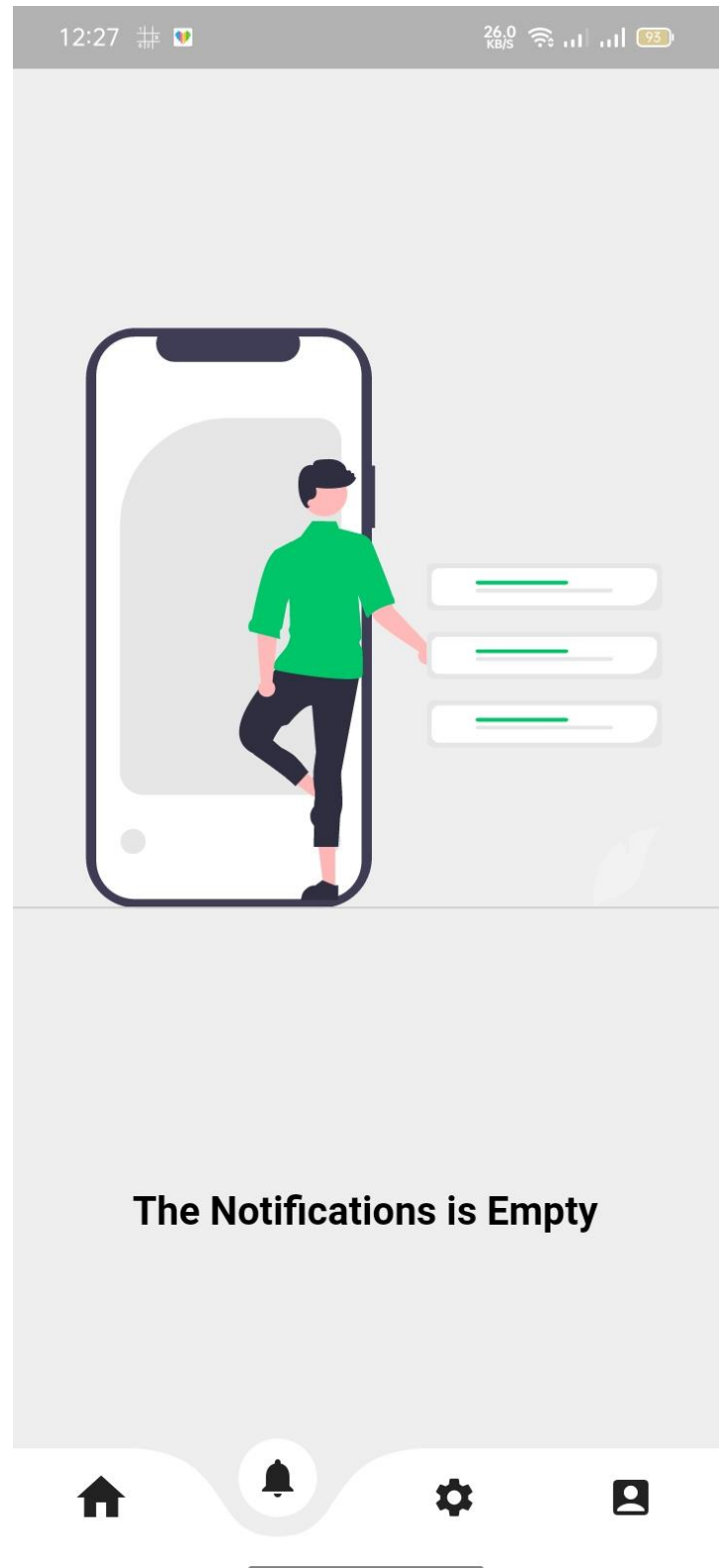
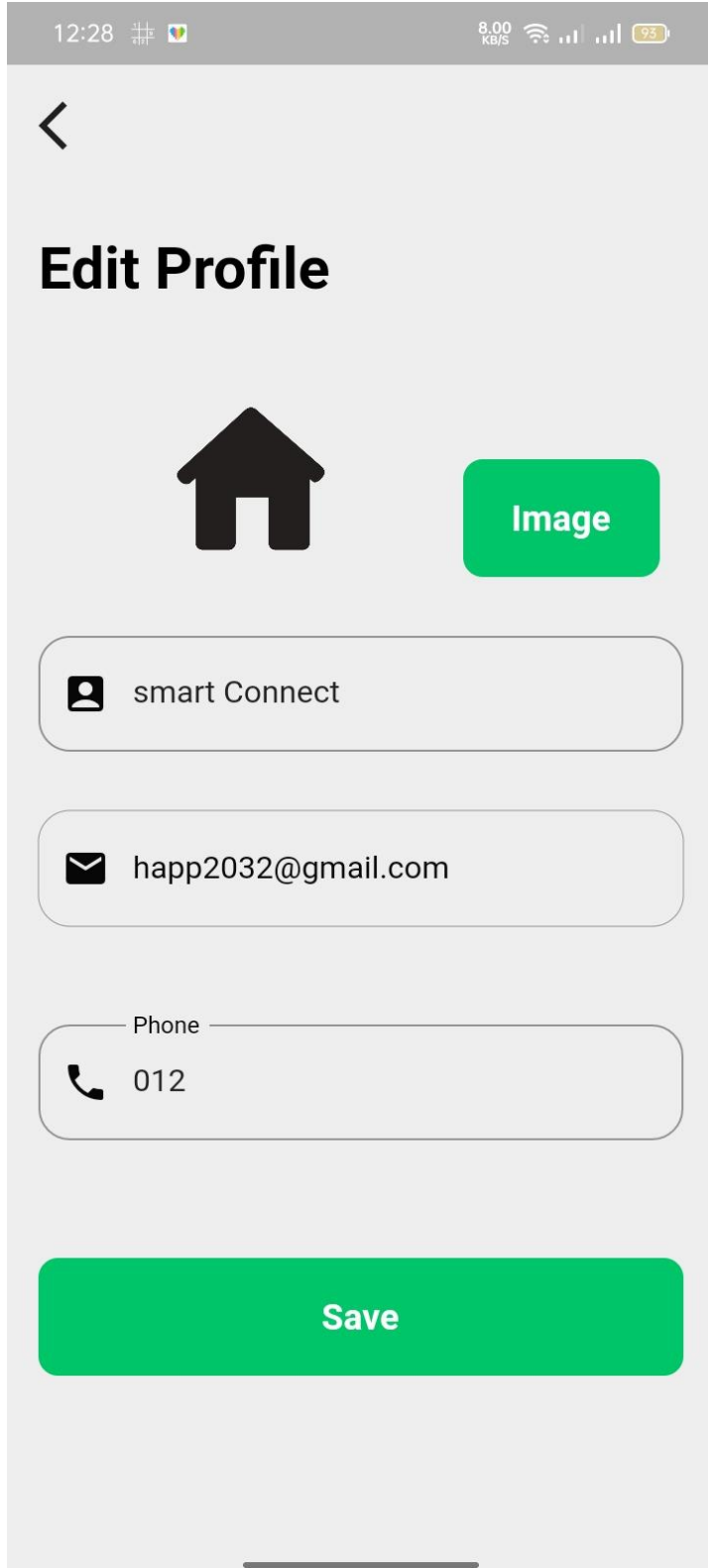


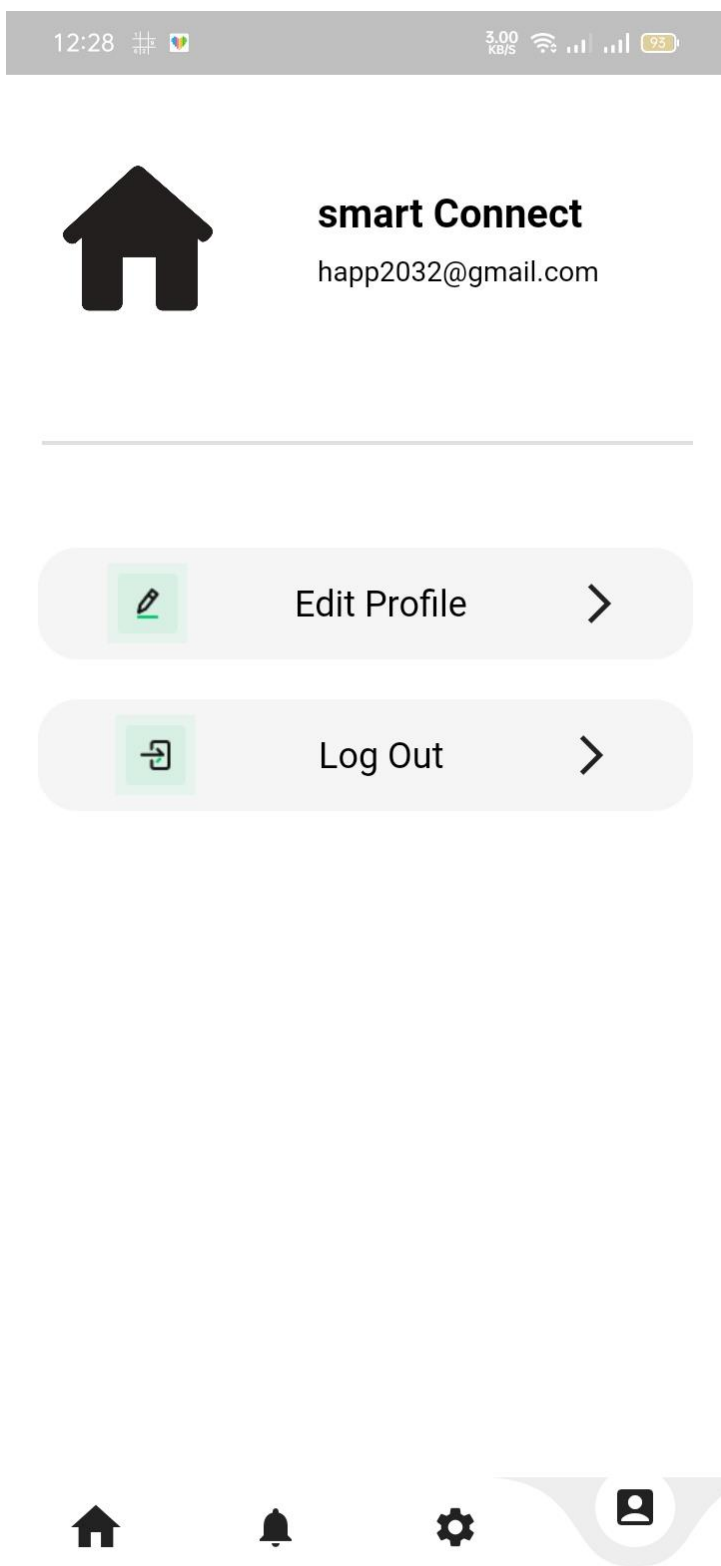
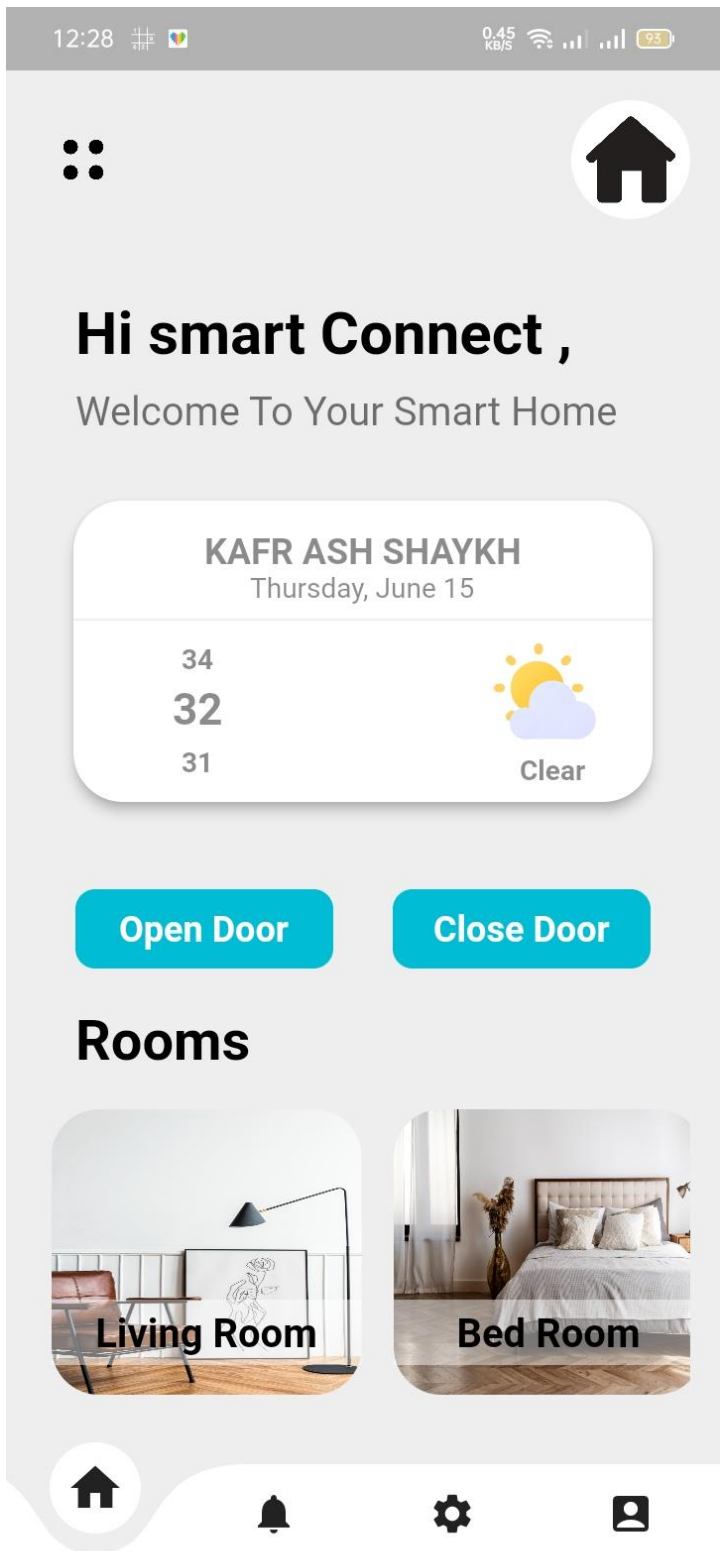
Connect To IOT Device



Password Of Door









 Scan Wifi

 **LinksysSMB5G 5G** **-61**
58:ef:68:b2:e5:ef

 **LinksysSMB24G** **-64**
58:ef:68:b2:e5:ee

 **LinksysSMB24G** **-76**
58:ef:68:b2:e5:e5

 **LinksysSMB24G** **-78**
58:ef:68:b2:e4:fb

 **LinksysSMB24G** **-79**
58:ef:68:b2:e6:c3

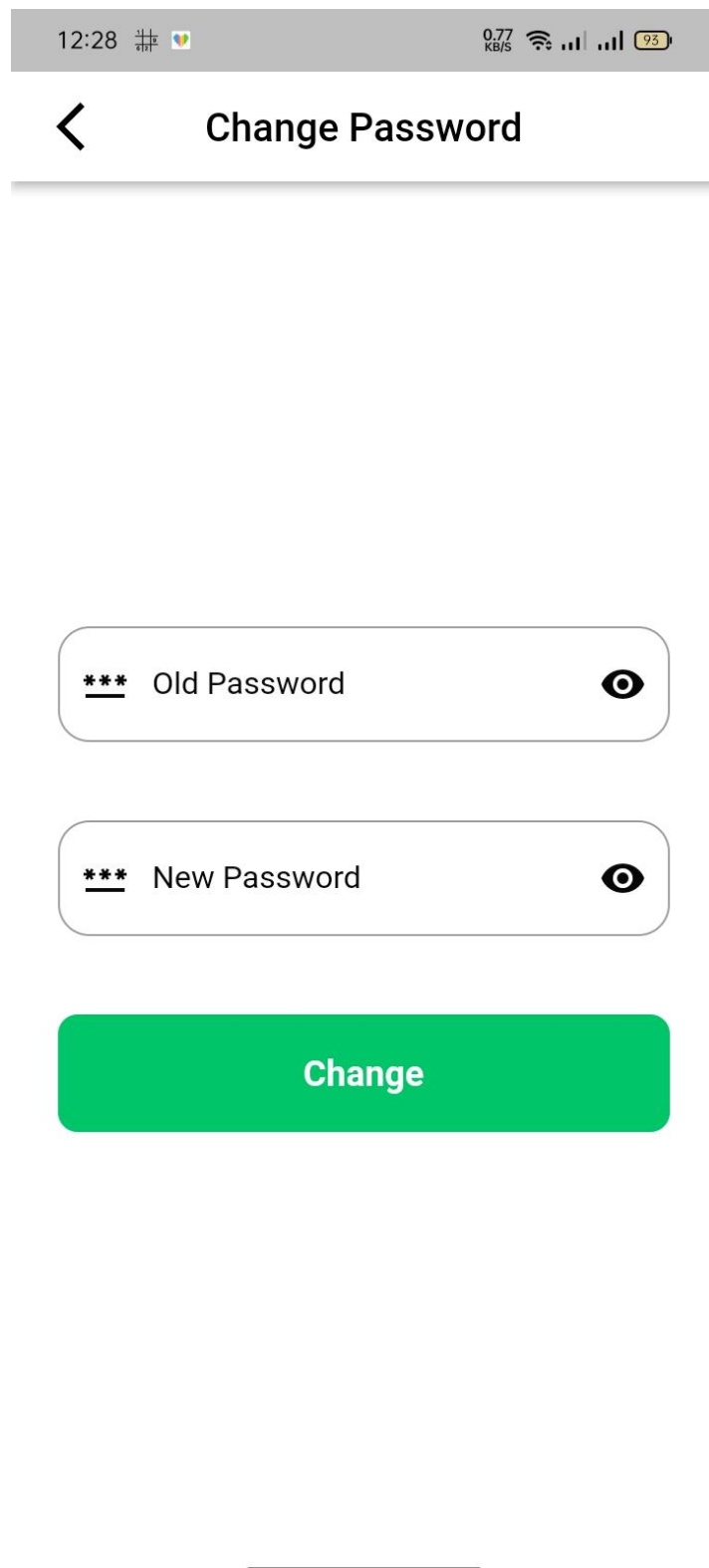
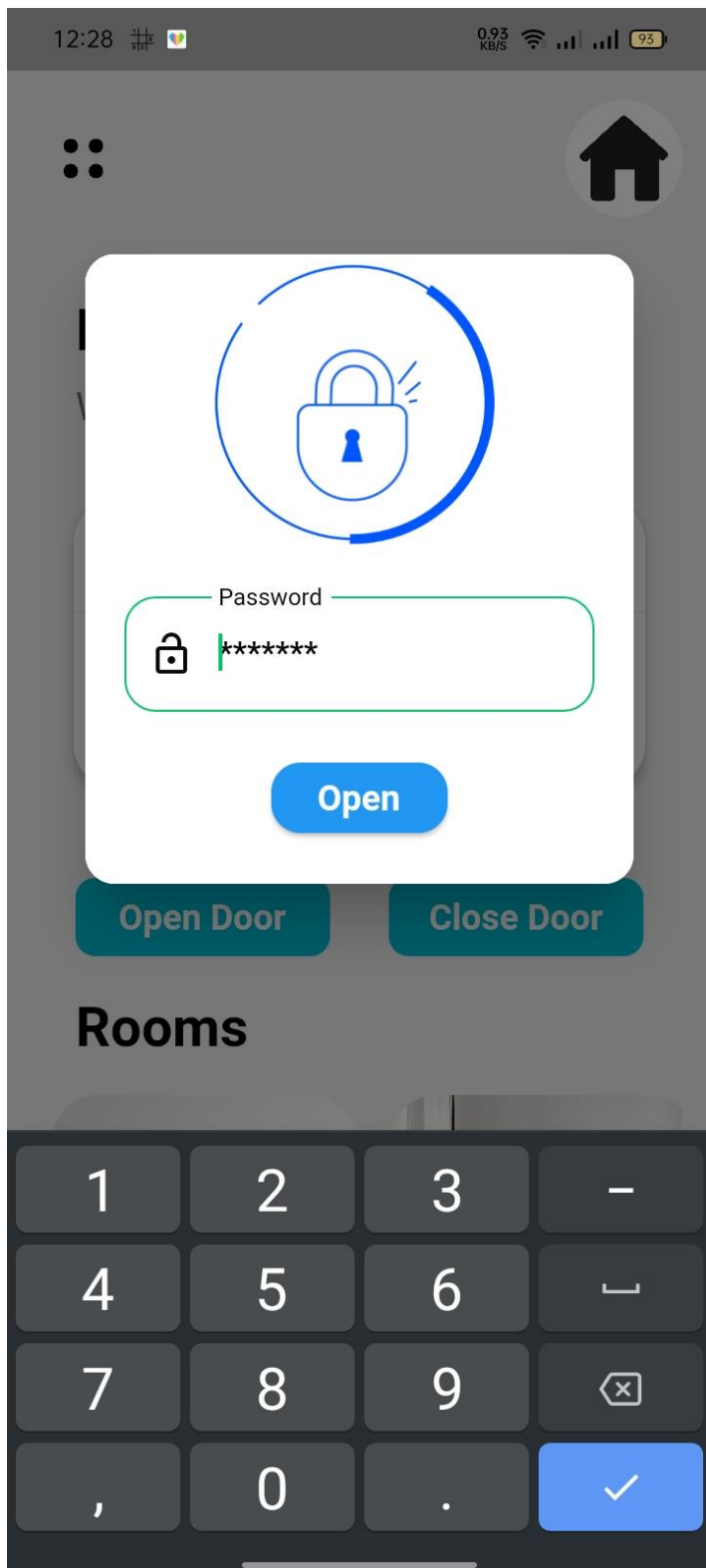
 Setup Connection

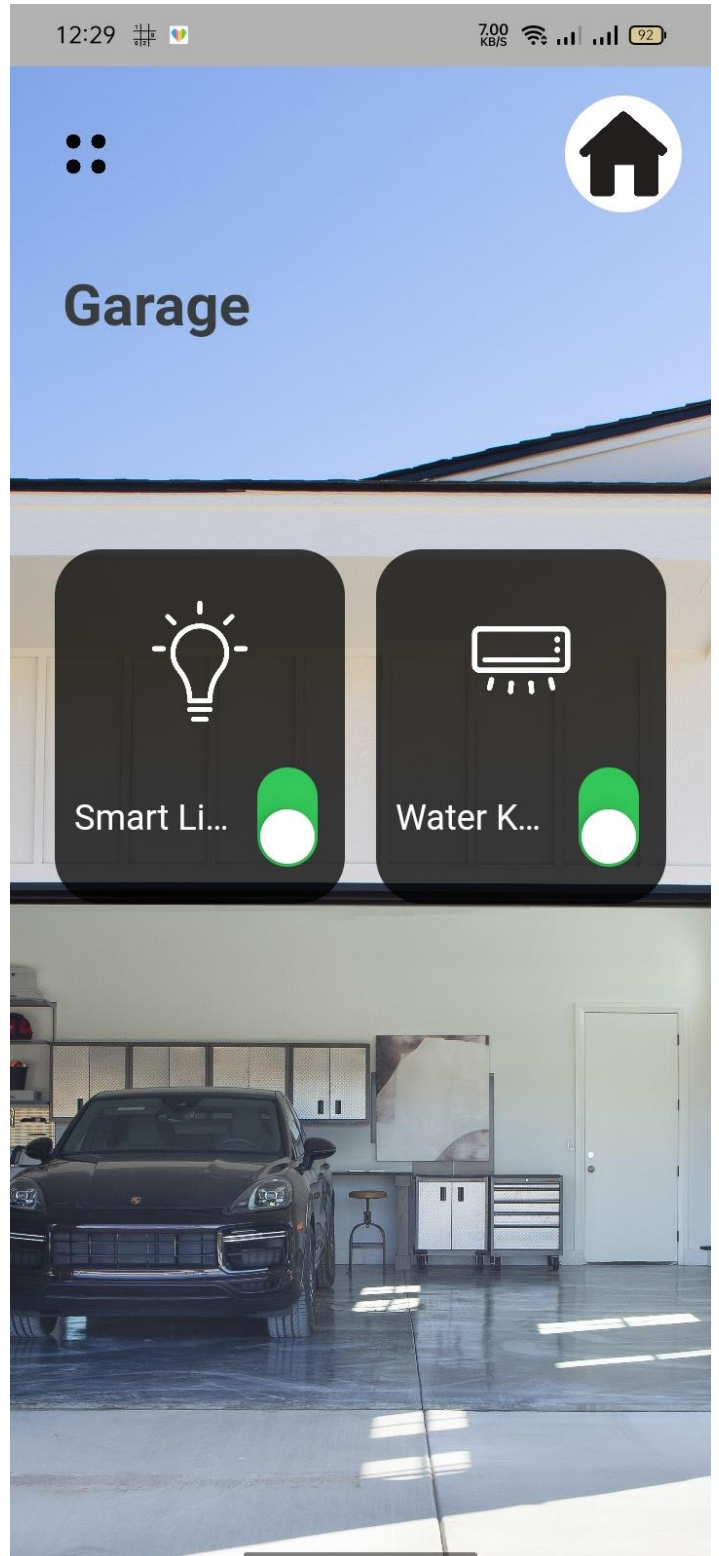
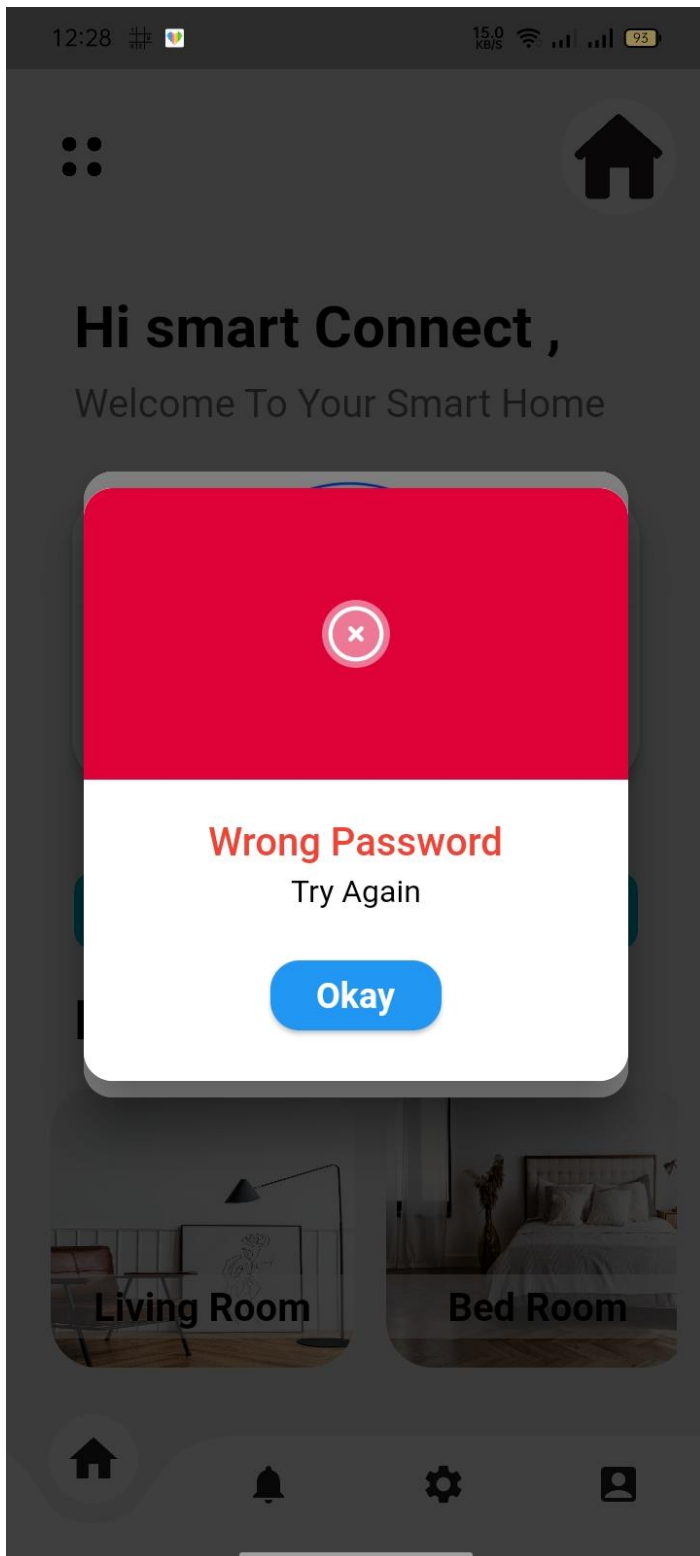
LinksysSMB5G

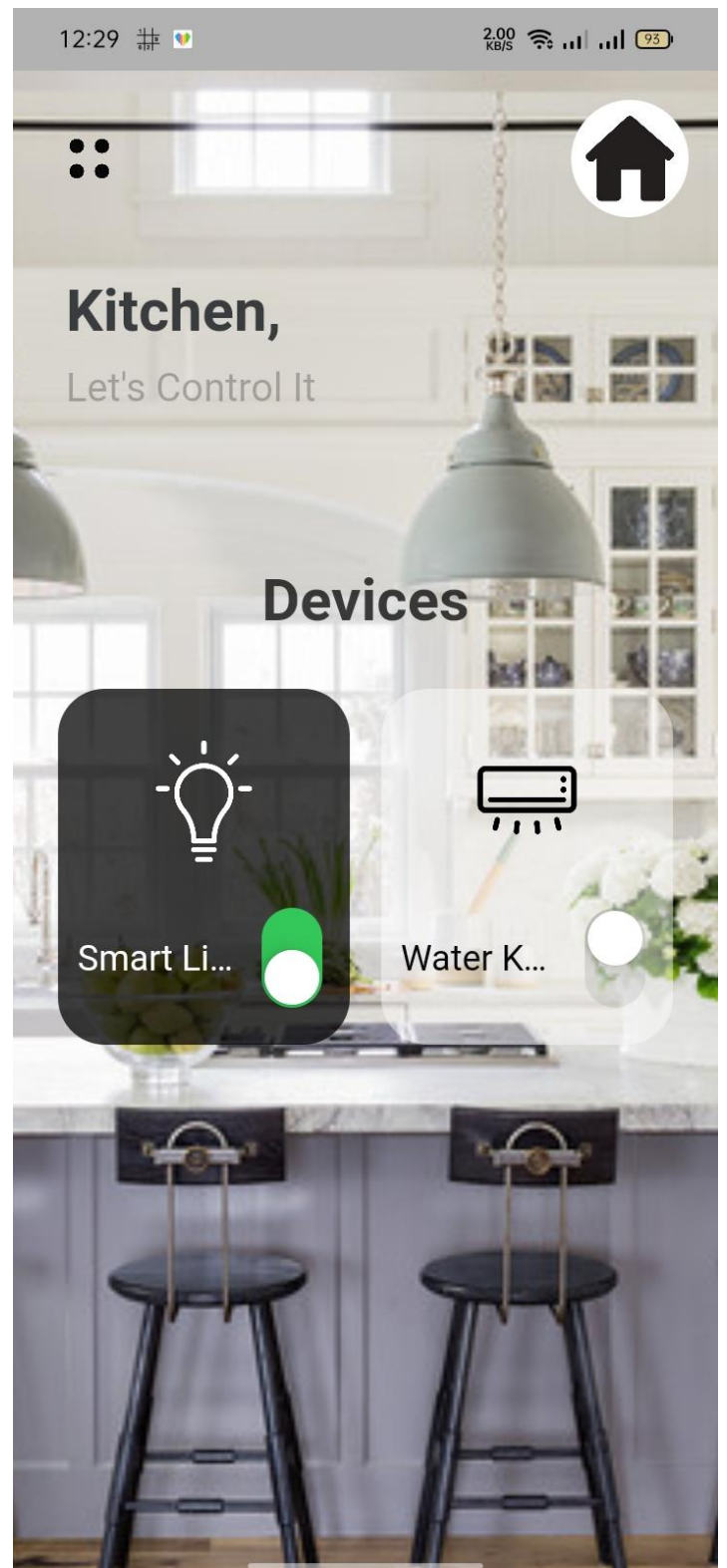
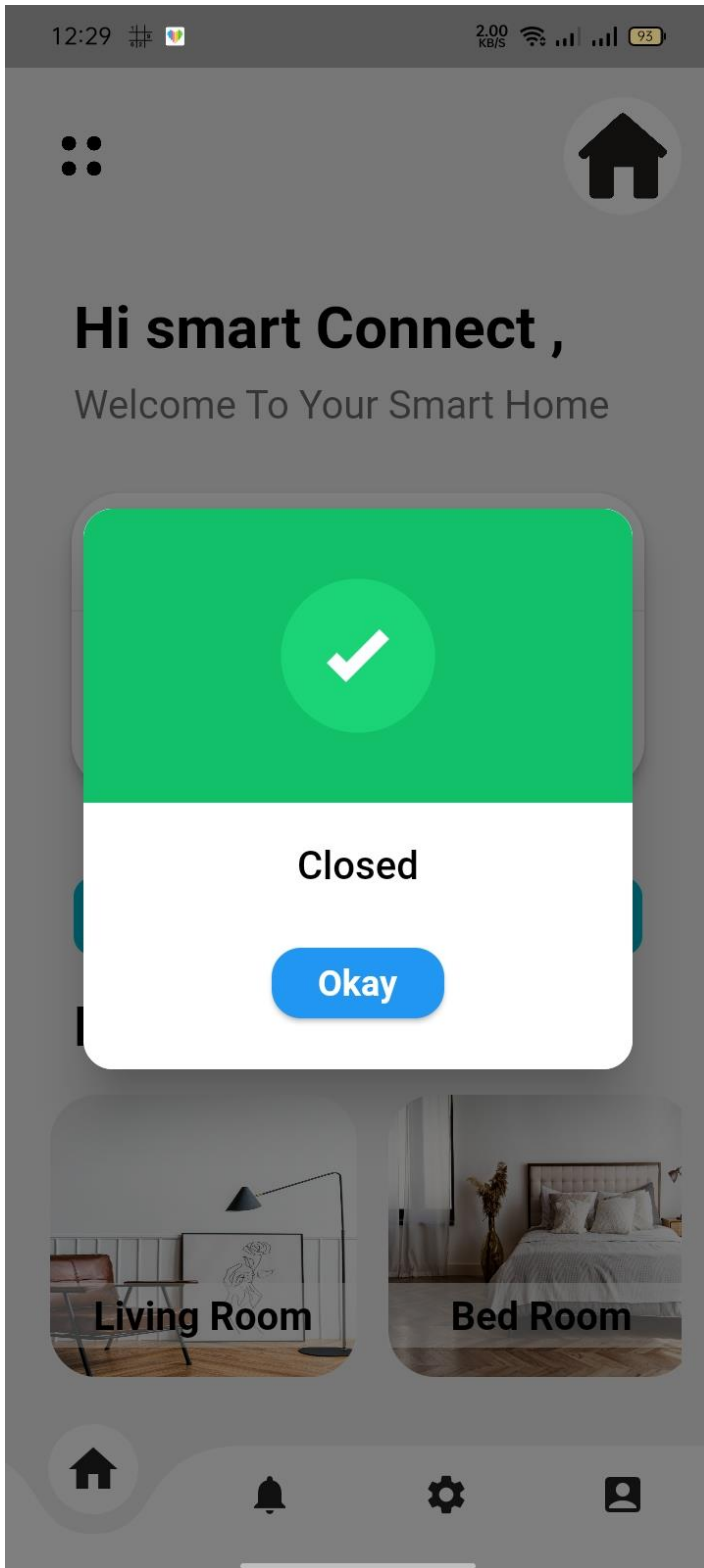
*** Password 

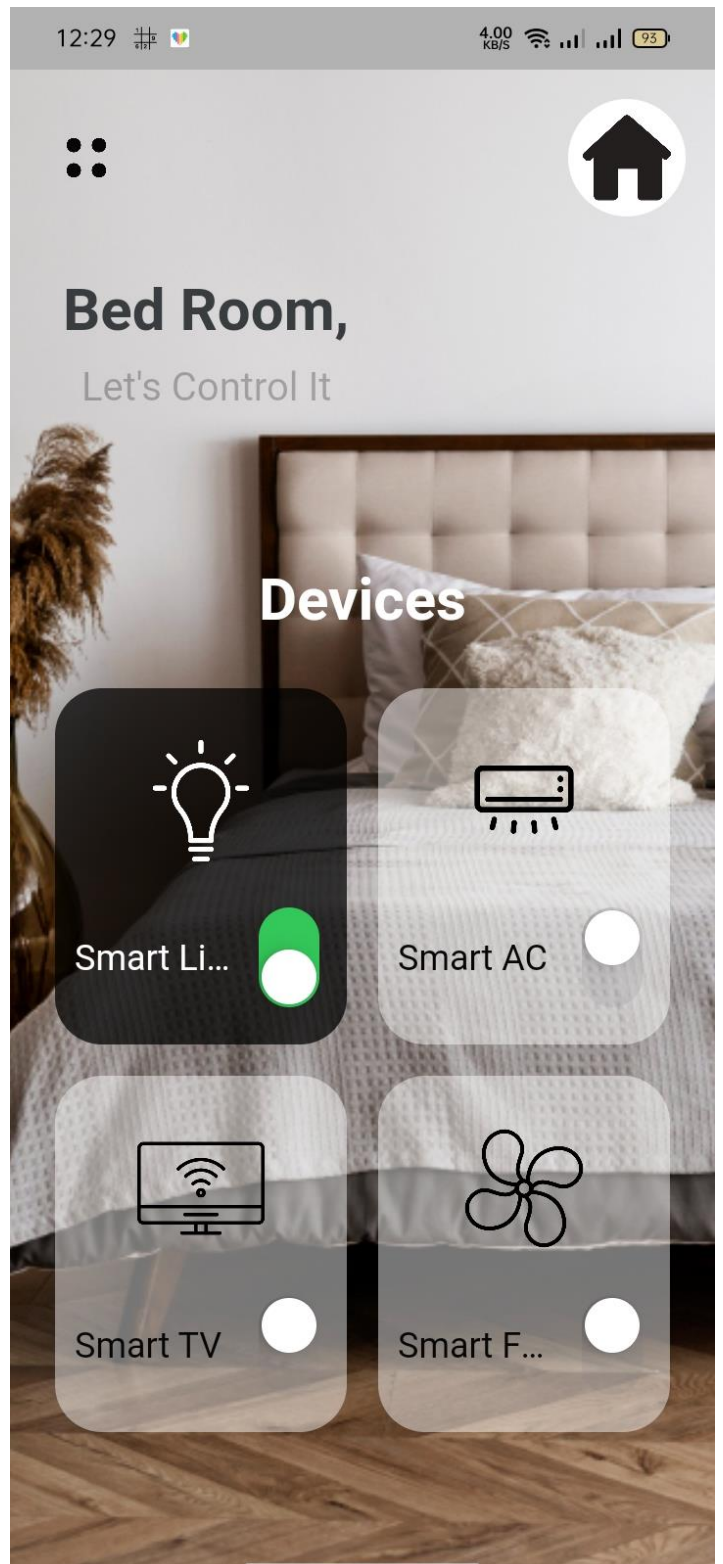
Connect

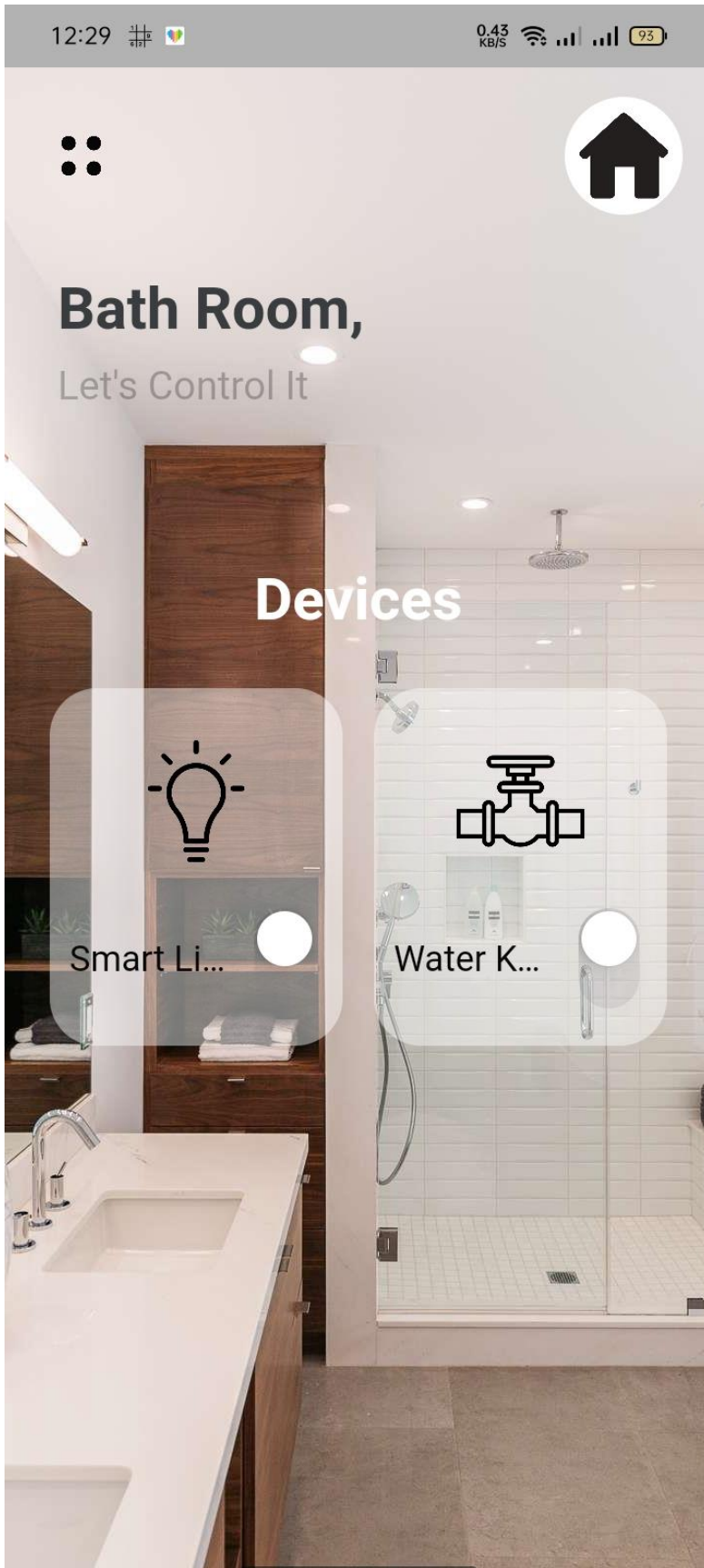


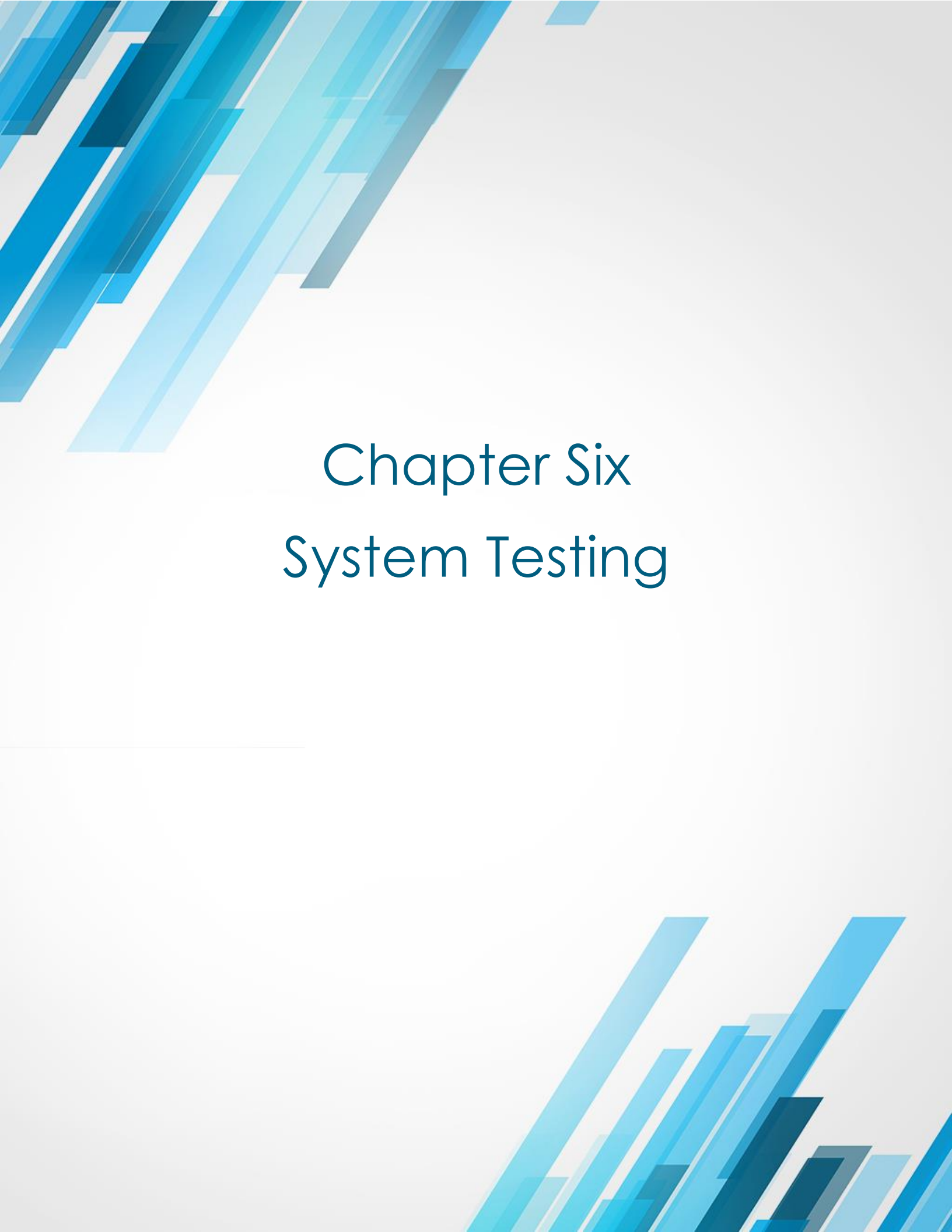










The image features a light gray background with abstract, overlapping blue geometric shapes in the top-left and bottom-right corners. These shapes consist of various shades of blue, from light to dark, arranged in a way that suggests movement and depth. The central text is positioned in the middle of the page, between these decorative elements.

Chapter Six

System Testing



6.1 Introduction

Smart home testing is a critical element of quality assurance and represents the ultimate review of specification, design and coding. In fact, testing is the one step in the embedded system process that could be viewed as destructive rather than constructive.

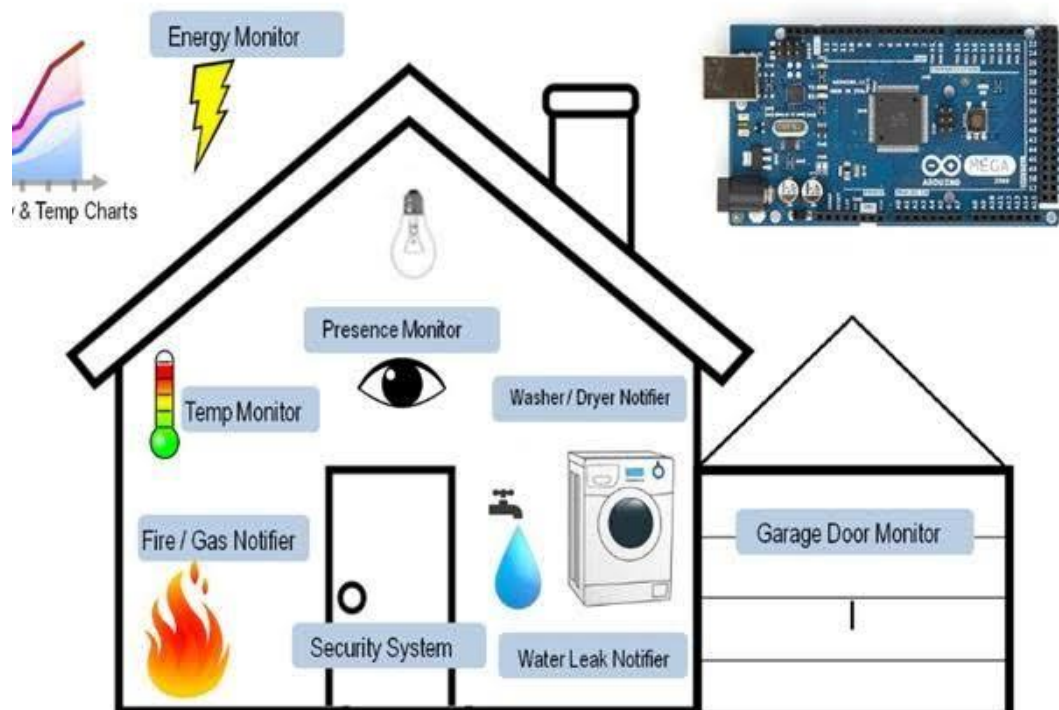
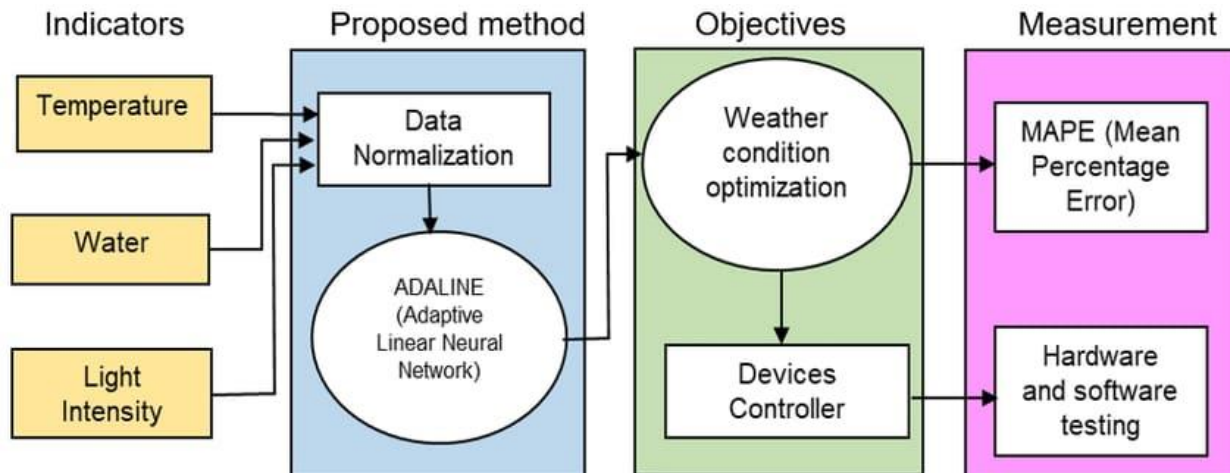
A strategy for Smart home testing integrates component test case design methods into a well-planned series of steps that result in the successful construction of Home. Testing is the set of activities that can be planned and conducted systematically. The underlying motivation of project testing is to affirm quality with methods that can economically and effectively apply to both strategic to both large and small-scale systems.

6.2 Strategic approach to software testing

We test every type of smart home devices on the market, including door locks, light bulbs, microwaves, plugs and power strips, security cameras, video doorbells, and many more. We install these devices in our actual homes, where we perform hands-on testing and compare them with similarly priced alternatives. We rate each product based on its features, interoperability with other smart products and standards, performance, and price.

The smart home market is flush with smart devices for energy management, home entertainment and appliances, home security, comfort and lighting. According to Counterpoint Research, more than 6 billion Wi-Fi capable smart home devices are expected to be shipped between 2020 and 2025. The smart home market is projected to have double-digit growth over the next several years, and with that comes the need for both hardware and software testing to ensure quality. Deliver quality with Applause's smart home testing solutions.

You've built a great piece of hardware, but will the app stand up? Test your smart home devices and deliver. Users demand effortless digital experiences from both smart home devices and their companion apps. 33% of consumers experience issues setting up or operating a connected device. 22% of those consumers return these devices for a refund. Hence, it's critical for the device manufacturers to develop a comprehensive testing strategy for verifying functional performance, as well as real user perspective on application flow, ease of installation and setup activation and customer service. Understand what your customers really want and deliver a user experience that your customers expect.





6.3 The levels of testing

ACCEPTANCE TESTING

Testing to verify a product meets customer specified requirements. A customer usually does this type of testing on a product that is developed externally.

FUNCTIONAL TESTING

Validating an application or Web site conforms to its specifications and correctly performs all its required functions. This entails a series of tests which perform a feature-by-feature validation of behavior, using a wide range of normal and erroneous input data. This can involve testing of the product's user interface, APIs, database management, security, installation, networking, etc. Testing can be performed on an automated or manual basis using black box or white box methodologies.

COMPATIBILITY TESTING

Testing to ensure compatibility of an smart home with hardware platforms. Compatibility testing can be performed manually or can be driven by an automated functional or regression test suite.

PERFORMANCE TESTING

Performance testing can be applied to understand your smart home, or to benchmark the performance. This sort of testing is particularly useful to identify performance. Performance testing generally involves an automated test suite as this allows easy simulation of a variety of normal, peak, and exceptional load conditions.

REGRESSION TESTING

Similar in scope to a functional test, a regression test allows a consistent, repeatable validation of each new release of a product or Web site. Such testing ensures reported product defects have been corrected for each new sensor and that no new quality problems were introduced in the maintenance process. Though regression testing can be performed manually an automated test suite is often used to reduce the time and resources needed to perform the required testing.





SMOKE TESTING

A quick-and-dirty test that the major functions of a piece of software work without bothering with finer details. Originated in the hardware testing practice of turning on a new piece of hardware for the first time and considering it a success if it does not catch on fire.

STRESS TESTING

Testing conducted to evaluate a component at or beyond the limits of its specified requirements to determine the load under which it fails and how. A graceful degradation under load leading to non-catastrophic failure is the desired result.

Light Bulbs

Testing smart light involves evaluating the installation process, light quality, connectivity options, integrations, and companion app controls. We start by downloading the mobile app, adding the bulb or light strip to the app's device list, and giving it a name. If we're testing a lighting kit, we install all of the bulbs in the package and assign them to a group.

For color bulbs, we test any preprogrammed color presets and create custom ones to see how the bulb performs and to gauge its color quality. We do the same for white color temperature bulbs to evaluate warm and cool temperature settings. We run several tests through the mobile app to see how responsive the bulb is (how quickly it responds to commands) and test voice commands if the bulb supports Alexa, Google, or Siri services. We also run routines or scenes to see how the bulb interacts with other smart devices and test its ability to follow usage schedules.



Locks

Testing smart locks can be a bit more involved than most other smart home devices, because we usually must remove an existing lock assembly in the process. Here, we look for things like well-documented installation instructions and thoughtfully designed mobile apps that walk you through the process.

We test the accuracy of their fingerprint scanners and the responsiveness of their touchpads, plus features such as voice controls, access schedules, auto-lock options, and geofencing. Additionally, we run Alexa and/or Google routines to see how the lock works with other smart devices such as cameras and lights.



Plugs and Power Strips

To test smart plugs, we start by downloading the companion app and creating a home account. Next, we evaluate the simplicity of the installation process. Most plugs require you to only pop them into a wall outlet. You can then add them to the app by pressing a pairing button or by connecting the device to your home network.

We test how well the plug responds to commands from the mobile app and the manual control button, as well as its ability to work with supported voice commands. If the plug works with other smart devices such as door locks, doorbells, and security cameras, we configure another device to trigger it via a routine or a scene and evaluate how seamlessly that process works.

We also create schedules to see how well the plug follows them. If it reports power usage, we compare the plug's energy readings to those from a competing plug or a power meter.

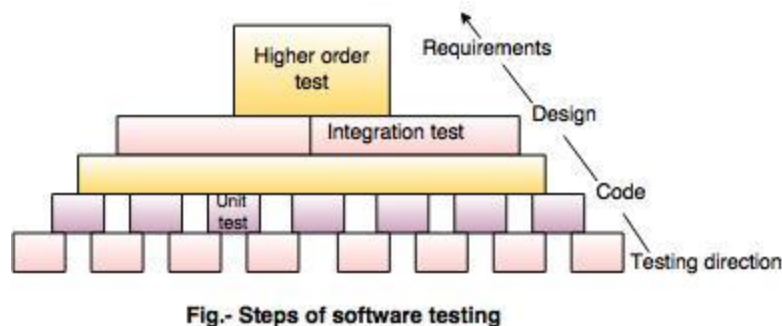


Figure 0-9 - Steps

6.5 Methodology

Steps to follow while implementing the methodology are as follows:

1. Start with a base function that you want to implement.
2. Create a document with the detailed requirement definition, an activity diagram with a description of the flow, database tables to be used, a component diagram, and a description of each component with the precondition and tables that would be affected by the component.



3. Give the document to the tester, and work with the tester while he or she writes the code to check if the steps in the document can be implemented and if the result of each use case can be achieved.

4. If the tester finds a step difficult to implement or thinks he or she is missing additional information to implement the functionality, then go to step 2; otherwise, go to step 3.

5. Ask the tester to log on to all the errors and difficulties he or she encountered while working on the prototype implementation.

6. Once the prototype is done and the results between the developer's prototype and tester's prototype match, work on the other requirement, and expand the prototype to final software.

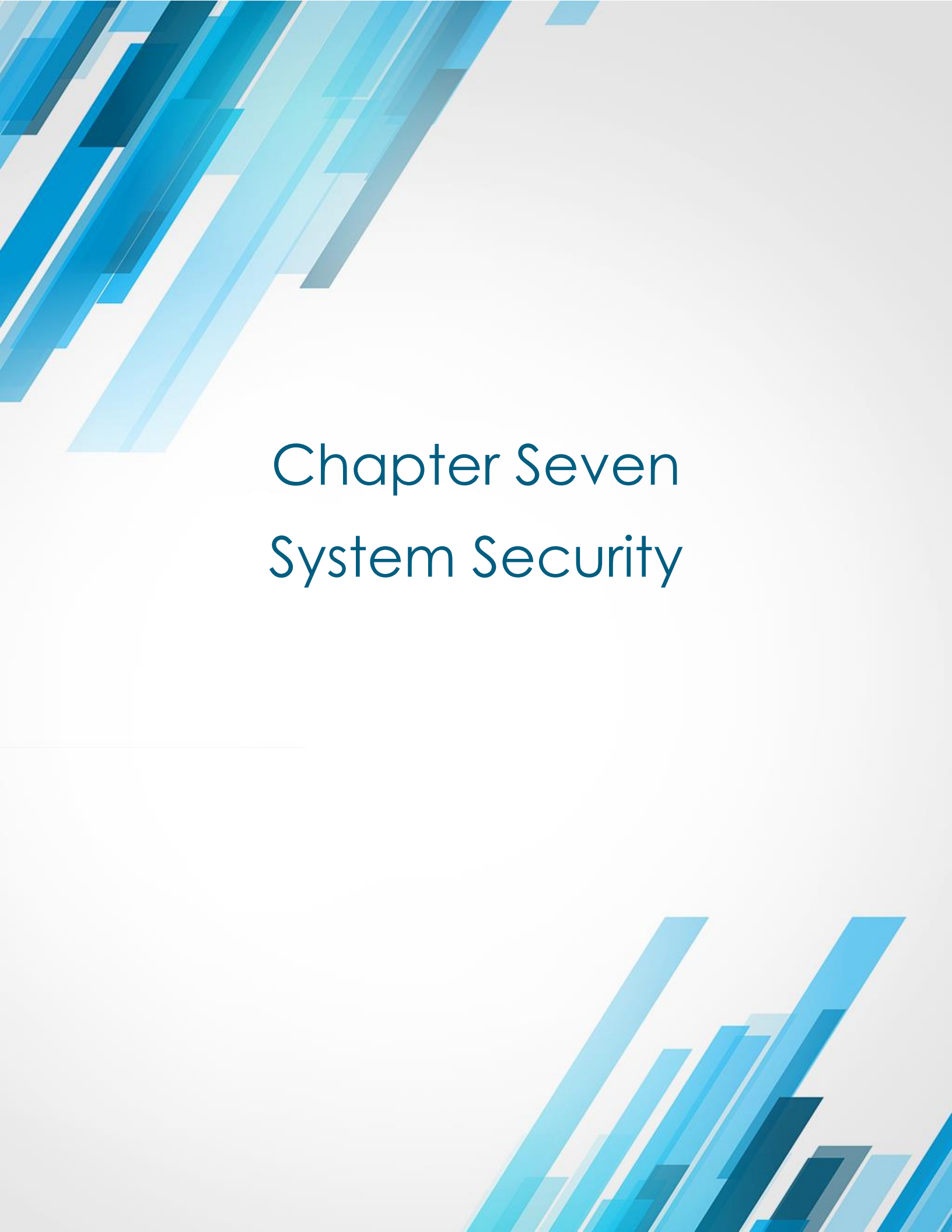
7. When the testing approach was implemented, the following pros and cons regarding the testing approach were realized.

Pros of using the methodology

- Helps give a better understanding of the requirements.
- Better design at the end of the cycle.
- Reduced testing to be performed at the end of the cycle - Documents produced would be of higher quality.

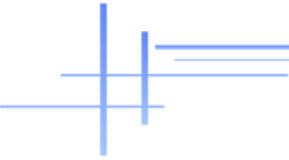
Cons of using the methodology

- The person working on the document should be experienced.
- There are increased time and money involved with testing.
- Different viewpoints for the same problem can lead to varying results.

The image features a light gray background with abstract, overlapping blue geometric shapes in the top-left and bottom-right corners. These shapes consist of various shades of blue, from light to dark, arranged in a way that suggests movement and depth. The central text is positioned in the middle of the page, between these decorative elements.

Chapter Seven

System Security



7.1 Introduction

The protection of computer based resources that includes hardware, software, data, procedures and people against unauthorized use or natural Disaster is known as System Security.

Home-based crime attacks, theft, and burglary are on the increase annually. Despite the lockdown and stay-at-home order, several homes were still attacked and burgled in South Africa.

According to the report in , attacks on homes and properties have aggravated into sexual offenses and assaults. This has shown a need for additional and improved levels of security for homes, properties, and individuals. The growth in the industrial development of smart home automation systems and the rate of research works carried out in the field give assurance of using the field to solve security issues arising in smart home environments.

A secured home will make occupants live in peace and without fear, irrespective of their location. Having remote control over the home, a clear view of the situation within the home, and a system that notifies the home owner of impending danger without any false alarm is of high importance in this era. Advancements in information technology (IT) and the internet of things (IoT) have provided platforms suitable for the control, monitoring, safety, and security of homes and properties.

The internet of things is the interconnection of several physical devices, network connectivity, and communication media to transmit information between devices and devices to humans. The IoT technologies have led to communication, interaction, and data exchange among devices, sensors, and appliances.

7.2.1 Client side validation

Various client side validations are used to ensure on the client side that only valid data is entered. Client side validation saves server time and load to handle invalid data.

Some checks imposed are:

- VBScript is used to ensure those required fields are filled with suitable data only. Maximum lengths of the fields of the forms are appropriately defined.
- Forms cannot be submitted without filling up the mandatory data so that



manual mistakes of submitting empty fields that are mandatory can be sorted out at the client side to save the server time and load.

Tab-indexes are set according to the need and taking into account the ease of user while working with the system.

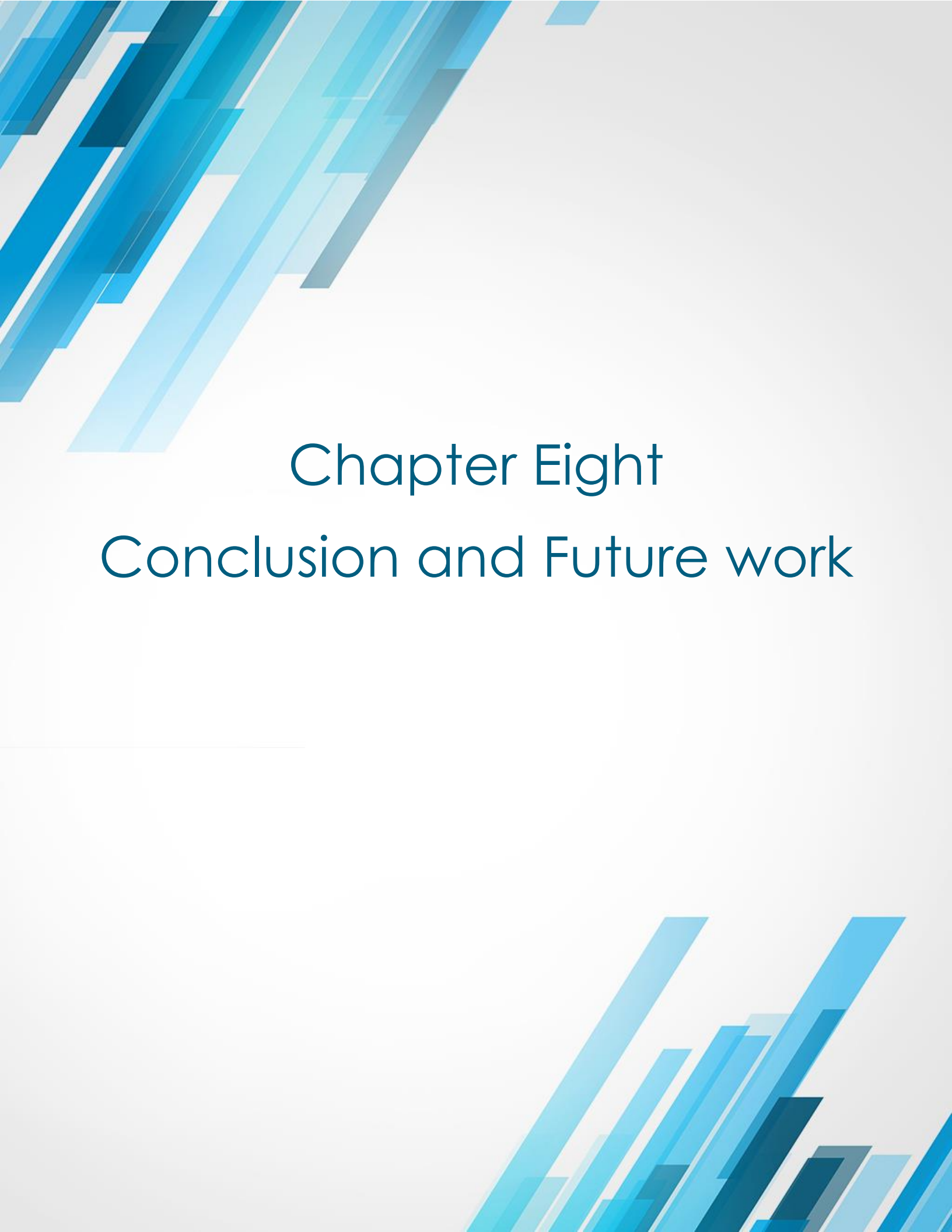
7.2.2 Server side validation

Some checks cannot be applied at client side. Server side checks are necessary to save the system from failing and intimating the user that some invalid operation has been performed or the performed operation is restricted.

Some of the server side checks imposed is:

- - Server side constraint has been imposed to check for the validity of primary key and foreign key.
- - A primary key value cannot be duplicated. Any attempt to duplicate the primary value results into a message intimating the user about those values through the forms using foreign key can be updated only of the existing foreign key values.
- - User is intimating through appropriate messages about the successful operations or exceptions occurring at server side.
- - Various Access Control Mechanisms have been built so that one user may not agitate upon another.
- - Access permissions to various types of users are controlled according to the organizational structure. Only permitted users can log on to the system and can have access according to their category.
- - User- name, passwords and permissions are controlled o the server side.
- - Using server side validation, constraints on several restricted operations are imposed.





Chapter Eight

Conclusion and Future work



8.1 Conclusion

The project has proposed the idea of smart homes that can support a lot of home automation systems. A smart home contains a connection between wireless communication, sensors, monitoring and tracking. Smart homes are a huge system that includes multiple technologies and applications that can be used to provide security and control of the home easily.

This project discussed the designed modules like sensors' circuits, monitoring and tracking of the home through IP camera, mobile notifications and home navigator.

In this project, an efficient approach for smart homes was proposed and implemented. C# programming language and ATMEL AVR microcontroller have been used to connect the sensors circuit to the home and to control the IP camera.

8.2 Future work

There are a variety of enhancements that could be made to this system to achieve greater accuracy in sensing and detection.

- a) There are a lot of other sensors that can be used to increase the security and control of the home like pressure sensor that can be put outside the home to detect that someone will enter the home.
- b) Changing the way of the automated notifications by using the GSM module to make this system more professional.
- c) A smart garage that can measure the length of the car and choose which block to put the car into it and it will navigate the car through the garage to make the parking easy for the homeowner in his garage.



8.3 Benefits of future work

1- One of the best things about the recent surge in popularity of smart home technology is how it has made houses more accessible for everyone. Before aging, individuals or those with physical disabilities might have had a harder time getting around their own homes but now with smart home systems, they can easily control everything in their house all from their fingertips! Not to mention, these sorts of systems also make houses more secure overall. So smart homes not only give people of all ages the opportunity to lead independent lives – they also give them peace of mind.

2-There are many benefits to having a smart home, such as convenience and efficiency. With a traditional home, you often have to schedule or control access to products manually. This can be time-consuming and inconvenient. A smart home automates many of these tasks, making your life easier.

3. Smart homes can help improve security by sending notifications or alerts to homeowners about potential dangers.

4. Smart homes offer multiple ways of performing a certain task, such as turning on lights manually or according to a schedule. This allows homeowners to have more control over their homes and how they use energy.

5. Although the initial investment for a smart home may be more expensive than a traditional home, the long-term cost savings as a result of efficient energy use will be worth it.

6. Did you know that having a smart home system can increase the value of your home when appraisers factor it in? Smart thermostats, smoke detectors, and home security systems have all been proven to do this! Not only that, but many insurance providers will also reduce your premiums if you have a smart system because it minimizes damage and lowers the number of claims that a policyholder will file. This is just one of the many ways that smart homes empower people of all ages to lead independent lives in their own homes!